

Redis 面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

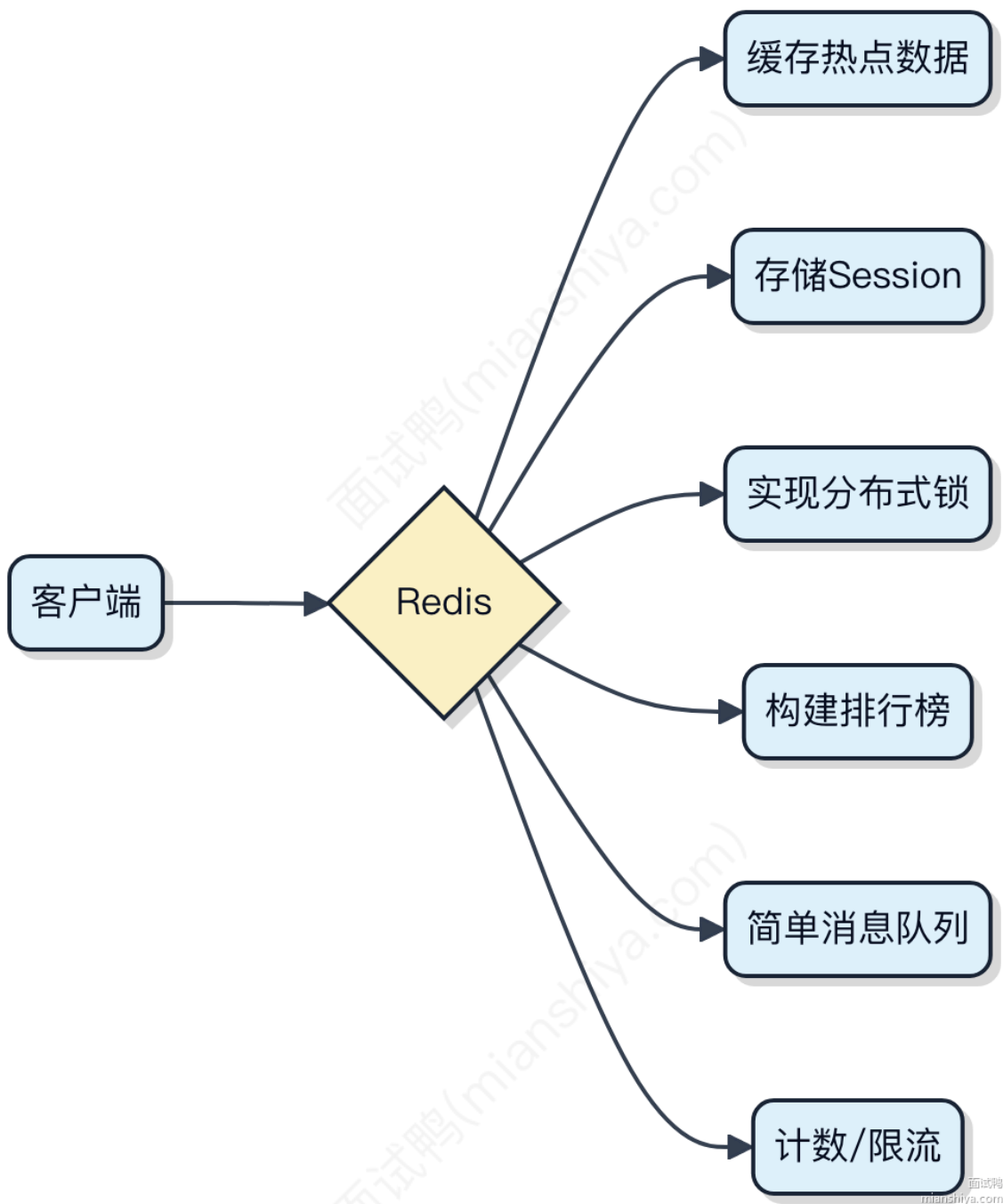
推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

Redis 通常应用于哪些场景?

缓存是最常见的用途, 把热点数据放在 Redis 里, 查询能从几十毫秒降到亚毫秒级。比如电商商品详情页, 用 Redis 能扛住每秒几十万的并发请求。

- 1) Session 存储也常用它, 像 Spring Session + Redis 可以实现分布式会话, 在集群环境下用户登录状态不丢失。
- 2) 分布式锁基于 SETNX 或 RedLock 实现, 多个服务节点协调执行, 避免资源冲突。注意锁过期时间设置要合理, 不然可能误删别人的锁。
- 3) 排行榜或计数场景特别适合, ZSet 支持按分数排序, 微博热搜、游戏积分榜基本都这么干。
- 4) 消息队列也能凑合用, 通过 List 结构做简单的生产消费, 但功能比不上 RocketMQ、Kafka 这种专业中间件, 复杂场景还是得换。
- 5) 限流经常结合 Lua 脚本做原子操作, 比如统计 1 分钟内接口调用次数, 超过阈值就拒绝请求, 保护后端服务不被搞崩。



持久化场景要谨慎，RDB 和 AOF 都有性能损耗，一般不拿它当主数据库用。临时数据、高频读写、需要快速响应的场景才是它的主战场。

Redis 中常见的数据类型有哪些？

Redis 支持几种核心数据结构，每种都有明确的使用场景。

字符串（String）是最基础的类型，能存文本、序列化对象，也常用来做计数器。比如用 `INCR` 实现接口调用次数统计，简单高效。

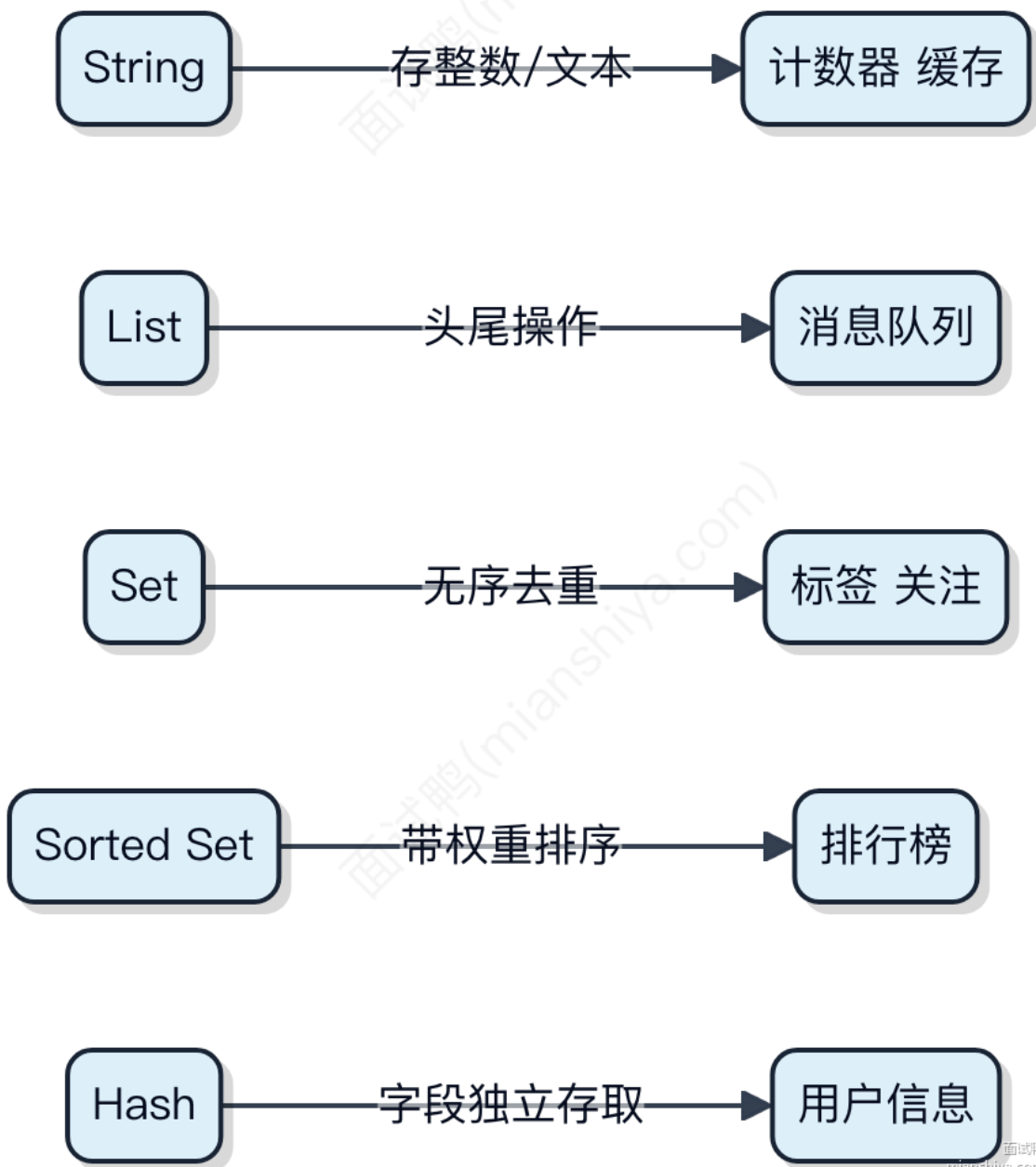
列表（List）底层是双向链表，支持从头尾插入删除，适合做消息队列。但随机访问慢，一般不推荐当数组用。
LPOP/RPOP 配合 LPUSH/RPUSH 能实现简单的生产消费模型。

集合（Set）是无序不重复的字符串集合，底层用哈希表实现，所以增删查都是 $O(1)$ 。可以用在用户标签管理、社交关系去重这些需要快速判断成员存在的场景。

有序集合（Sorted Set）在 Set 基础上加了 score 排序，比如排行榜类需求，ZADD 加数据，ZRANGE 按排名取，性能很好。score 可重复，member 才是唯一键。

哈希（Hash）适合存储对象字段，比如用户信息，field 对应属性名，value 存值。相比直接序列化整个对象，可以单独更新某个 field，省带宽也省内存。

这五种是日常最常用的，像 Stream、Geo、Bitmap 这些属于特定场景扩展类型，用得相对少一些。



Redis Zset 的实现原理是什么？

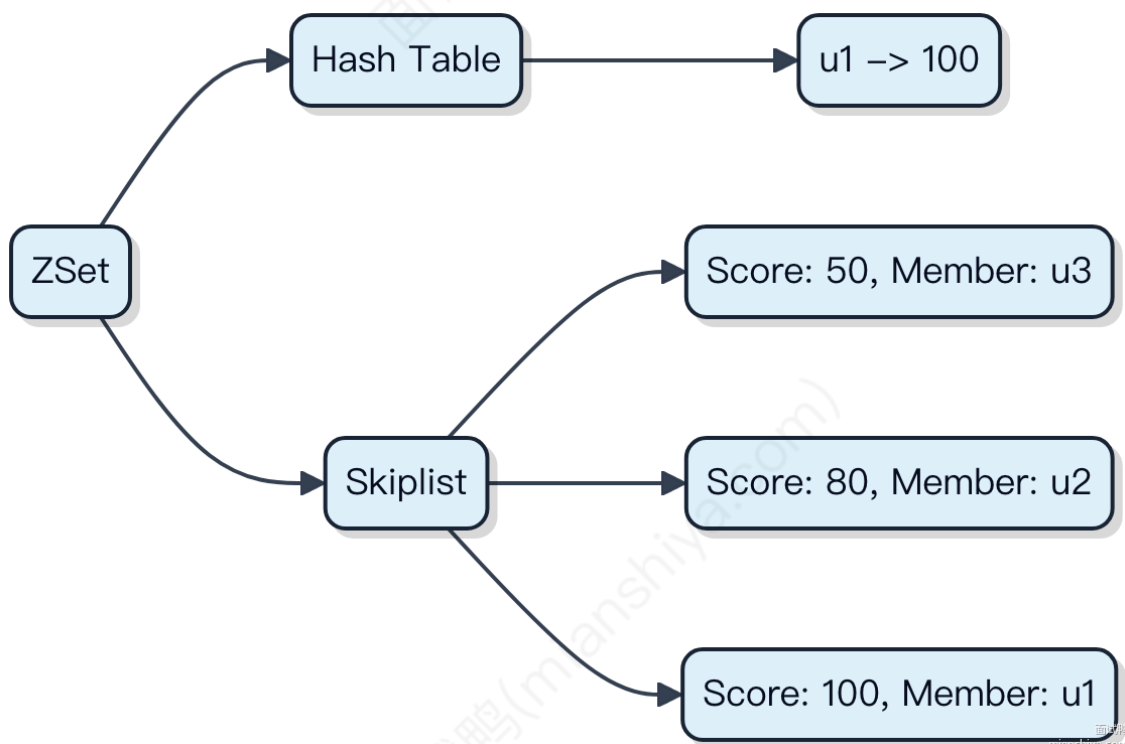
Redis 的 Zset 为了适应不同场景，在底层用了两种编码实现，完全基于元素数量和大小动态切换。

当数据量小、成员长度短时，用 `ziplist` 存储。它把 `score` 和 `member` 连续存放在一段内存里，节省空间，插入删除快。一旦元素超过 `zset-max-ziplist-entries`（默认 128），或者任一 `member` 字符串长度超过 `zset-max-ziplist-value`（默认 64 字节），就转成 **跳表（skiplist）** 加哈希表的组合。

跳表是核心，解决有序问题。它有多层链表，最底层按 `score` 升序存所有元素，上层是索引，每次查询像“电梯式”快速下落，时间复杂度稳定 $O(\log N)$ 。Redis 的 `skiplist` 是单向后指针，但带反向遍历能力，支持 `ZREVRANGE`。

哈希表也不能少，它保存 `member` 到 `score` 的映射，确保 `ZSCORE` 查询是 $O(1)$ ，避免在跳表里线性查找。

所以最终结构是：一个 `zset` 对象同时包含一个跳表和一个哈希表，空间换时间。比如你在做排行榜，`ZADD user:ranking 100 u1`，`member` 和 `score` 会同时写入这两个结构。



这种设计在性能和内存间做了折中，小数据用 `ziplist` 省内存，大数据用 `skiplist` 保效率。你一般不用关心切换过程，Redis 自动完成。

你在项目中使用的 Redis 客户端是什么？

项目里用 Redis 客户端，一般首选 **Lettuce**，尤其是 Spring Boot 2.0 之后的项目。它基于 Netty 实现，支持响应式编程，连接是长连接，多个线程可以共享一个连接，资源利用率高。

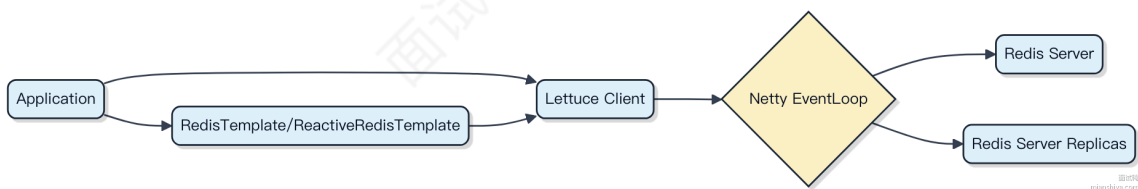
Jedis 也用过，早期项目常见，但它默认是同步阻塞的，每个操作都独占一个连接，高并发下连接数容易打满。除非是特别轻量的场景，不然现在基本不选它了。

Lettuce 和 Spring Data Redis 配合起来特别顺手，配置简单，API 也清晰。比如用 `ReactiveRedisTemplate` 做响应式缓存，或者普通 `RedisTemplate` 操作字符串、哈希都没问题。

```
@Bean
public RedisConnectionFactory redisConnectionFactory() {
    return new LettuceConnectionFactory(new
RedisStandaloneConfiguration("localhost", 6379));
}
```

遇到过一次连接泄漏，是因为用了 Lettuce 的异步 API 但没处理好 Future 回调，导致 Netty 线程堆积。后来统一用 `RedisTemplate` 封装，避免直接操作底层连接。

1) 生产环境一定要配连接池，Lettuce 虽然原生无连接池，但可以通过 `LettucePoolingClientConfiguration` 启用。2) 大 key 或频繁批量操作要控制粒度，避免阻塞主线程。3) 开启 SSL 和设置超时时间是上线前的必备动作。



Redis 字符串类型的最大值大小是多少？

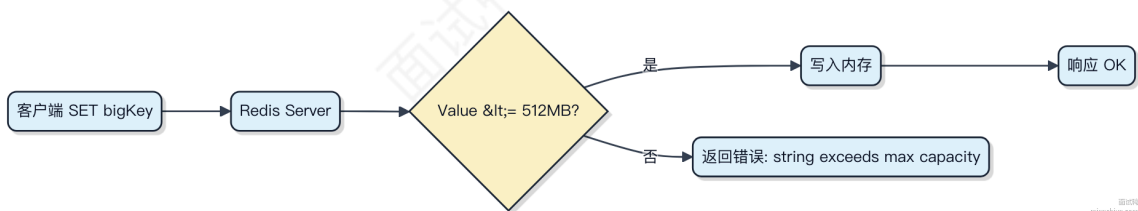
Redis 的字符串类型最大能存 512 MB。

这个限制是硬编码在 Redis 源码里的，不是随便定的。你往一个 key 里塞超过 512 MB 的数据，Redis 直接拒绝，返回错误。这么做主要是为了防止单个 value 太大，把内存打爆，或者在网络传输时卡住整个实例。毕竟 Redis 是单线程处理命令的，一个超大 value 的读写可能让其他请求全部排队，延迟飙升。

一般情况下，我们也不会用 Redis 存这么大的数据。比如缓存网页内容、JSON 数据，通常都在几 KB 到几十 KB。真要存大文件，比如图片、视频，通常做法是把文件放在对象存储（比如 S3、MinIO），Redis 只存个 URL 或索引。

如果你看到有人把序列化后的大型对象塞进一个 key，那基本是设计问题。拆分结构、用哈希或者换存储更合适。

需要特别注意的是，虽然最大支持 512 MB，但生产环境建议单个 value 控制在 10 KB 以内。超过 100 KB 就算大 key 了，容易引发慢查询和内存不均。



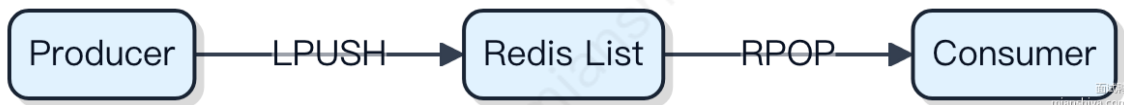
如何在 Redis 中实现队列和栈数据结构？

Redis 本身不直接提供队列和栈的类型，但它的 List 结构天生适合做这个事。你用 `LPUSH` 和 `RPOP` 就能实现一个先进先出的队列，反过来 `LPUSH` 和 `LPOP` 就是栈的后进先出。

1) 实现队列时,生产者用 `LPUSH` 往左边塞数据,消费者用 `RPOP` 从右边取。这样最早进的数据在链表尾部,最先被消费。如果担心 `RPOP` 在没数据的时候卡住,可以换成 `BRPOP`,设置个超时时间,比如 `BRPOP queue_key 5`,最多等 5 秒。

2) 实现栈就更简单了, `LPUSH` 和 `LPOP` 都操作头节点,压栈和弹栈都在同一边,天然符合栈的行为。

不过实际项目里,光靠这两个命令可能搞不定复杂场景。比如消息丢了怎么办?这时候就得上 **Redis Streams**,这是 Redis 5.0 加的功能,专门用来做消息流处理。Kafka 是重量级选手,如果你系统里已经用了 Redis,那 Streams 基本就是首选,轻量又支持持久化、回溯、消费者组。



注意一点, List 做队列时别用 `LRANGE` 自己轮询,太耗资源。让客户端阻塞等待才是正道。还有,如果需要确保消息不丢,记得配持久化策略,不然重启就没了。

Redis String 类型的底层实现是什么? (SDS)

Redis 里的 String 类型底层其实用的不是 C 语言原生的 `char` 数组,而是自己搞了一套叫 **SDS** (Simple Dynamic String) 的数据结构。这么做主要是为了解决 C 字符串在实际使用中的痛点,比如获取长度得遍历、没法安全拼接、容易缓冲区溢出这些。

SDS 的结构很简单,核心就三个字段: `len` 记当前用了多少字节, `free` 记后面还剩多少空闲空间, `buf[]` 是真正的字符数组。因为存了 `len`,所以 `strlen` 这种操作是 $O(1)$ 的,压根不经过遍历。拼接字符串前它会先看 `free` 够不够,不够就扩容,避免了溢出问题。

还有个关键点是 SDS 在修改时会做空间预分配和惰性释放。比如你追加内容,它不会刚好分配你需要的大小,而是多分一点,下次再改就能省掉一次内存调整。删数据时也不立刻 `free`,留着 `free` 字段里,方便后续复用。

代码长这样:

```
struct sdshdr {
    int len;
    int free;
    char buf[];
};
```

像 Redis 做 RDB 写入或网络传输时,经常要二进制安全地处理数据,SDS 不依赖 `\0` 结尾,能存任意字节,这也是比 C 字符串强的地方。

如何使用 Redis 快速实现排行榜?

用 Redis 做排行榜,核心是利用 **Sorted Set** 这个数据结构。它天然支持按分数排序,增删改查都是 $O(\log N)$,性能完全够用。

1) 每个玩家的得分用 `ZADD leaderboard {score} {player}` 写入。后续更新直接覆盖同名 member,Redis 会自动按新分排序。

2) 查 Top 10 直接 `ZRANGE leaderboard 0 9 WITHSCORES`,从高到低取前 10。如果要倒序(比如末尾排名),用 `ZREVRANGE`。

3) 想查某个玩家的排名, `ZSCORE leaderboard player` 拿分数, 再用 `ZRANK leaderboard player` 拿正序名次, 或 `ZREVRANK` 拿倒序。

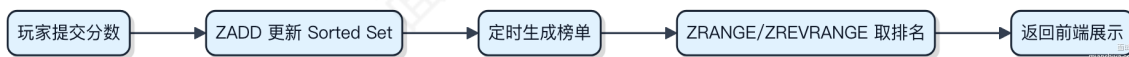
实际场景里, 像游戏积分榜、直播热度榜、电商销量排行, 基本都这么干。Redis 的 Sorted Set 在内存里维护跳跃表, 查询快得飞起, 写入也高效。

代码示例:

```
// 添加或更新分数
jedis.zadd("leaderboard", 95.5, "player1");

// 获取 Top 10
Set<Tuple> top10 = jedis.zrevrangeWithScores("leaderboard", 0, 9);
```

注意点: 数据量特别大时, 比如千万级用户, 单个 zset 可能撑不住。可以按维度拆, 比如分服排行榜, 或者用 zset + 分页缓存组合方案。另外别忘了设过期时间, 避免数据堆积。



如何使用 Redis 快速实现布隆过滤器?

布隆过滤器的核心是用多个哈希函数和位数组判断元素是否存在, Redis 本身不直接支持位操作的复合结构, 但可以用 **HyperLogLog** 或 **Bitmap** 搭配外部逻辑来实现。实际项目里更常见的是用 Redis Module, 比如 **RedisBloom**。

1) 先说原理。布隆过滤器往里加数据时, 会算出多个哈希值, 对应到位数组的不同位置, 把这些位设为 1。查的时候看这些位是不是都是 1, 有一个是 0 就说明肯定不存在, 全是 1 则大概率存在 (有误判率)。

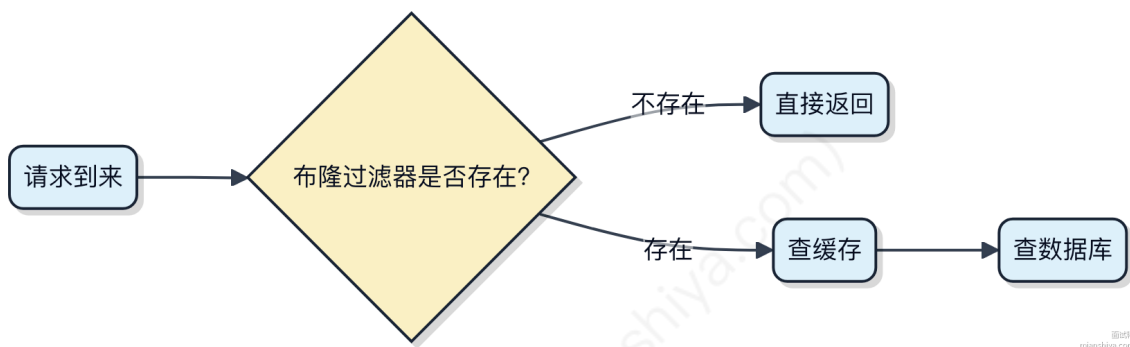
2) 在 Redis 里搞这个, 最省事的是加载 RedisBloom 模块。它提供了 `BF.ADD`、`BF.EXISTS` 这种命令, 直接就能用。比如:

```
BF.ADD user:exists "user123"
BF.EXISTS user:exists "user456"
```

背后 RedisBloom 自己管理位数组和哈希函数, 你不用操心细节。

3) 如果不允许装模块, 就得自己用 `setbit` 和 `getbit` 模拟。比如用 `key + offset` 表示某个哈希位置, 自己写逻辑算 `n` 个哈希值。但要注意控制 `key` 的粒度, 别让单个 `key` 太大, 不然搞不定。

典型场景是缓存穿透防护。像商品详情页查 DB 前, 先过一遍布隆过滤器, 99% 的无效 ID 被挡掉, 数据库压力直接降下来。



如何使用 Redis 统计大量用户唯一访问量 (UV) ?

用 Redis 做海量用户 UV 统计，直接存用户 ID 肯定搞不定，内存撑不住。比如 1 亿用户，每个 ID 存成字符串也得上百 MB 到 GB 级别，更别说重复访问还得去重。

这时候就得上 **HyperLogLog**。它是一种概率数据结构，专门用来做基数估算——也就是“不精确但误差可控”的去重统计。它的神奇之处在于，不管你是 10 万还是 1 亿用户，Redis 里只占 **12KB** 内存。

底层原理是通过哈希函数把元素映射成二进制串，利用“前面连续零的个数”来估算整体分布，再用调和平均减少误差。官方说标准误差是 0.81%，对大多数业务完全够用，比如活动页面、广告曝光这些场景。

代码用起来也简单：

```
// pfadd key element
redisTemplate.opsForHyperLogLog().add("uv:page1", "user123");
// pfcount key
long uv = redisTemplate.opsForHyperLogLog().size("uv:page1");
```

还能合并多个统计值：

```
// pfmerge destkey sourcekey [sourcekey ...]
redisTemplate.opsForHyperLogLog().union("uv:total", "uv:page1", "uv:page2");
```

注意点：HyperLogLog 不支持取出具体元素，也不能获取实时精确值。如果非要精确去重，还是得靠布隆过滤器 + Set 或者下沉到 Flink 实时计算。

对于日活这种量级，单个 HLL 12KB，一天一个 key，一年也就 4MB 左右，非常轻量。



Redis 中的 Geo 数据结构是什么？

Redis 的 Geo 结构本质上是基于 **Sorted Set** 实现的，用 zset 来存地理位置的坐标，把经纬度编码成一个 score，查的时候靠范围查询搞定。

- 1) 存的是某个地点的经度、纬度，以及一个名称（比如“北京西站”），实际存储时会把经纬度用 Geohash 编码转成一个 52 位整数作为 zset 的 score。
- 2) 支持的核心命令有 GEOADD、GEODIST、GEORADIUS 等。比如 GEORADIUS city 116.4 39.9 10 km 就是查坐标附近 10 公里内的所有点。
- 3) 底层其实是 zset，所以可以用 ZRANGE 或 ZREM 直接操作，但一般不建议绕过 geo 命令。

适合做附近的打卡店、共享充电宝、网约车这些场景，像美团、滴滴早期都拿 Redis Geo 做过就近匹配。

有个坑要注意：经纬度不能超范围，经度 ~~-180+180~~，纬度 ~~-85.0511+85.0511~~，超出会报错。另外精度问题也得注意，Geohash 本身有边界误差，跨格子可能漏点。

代码示例：

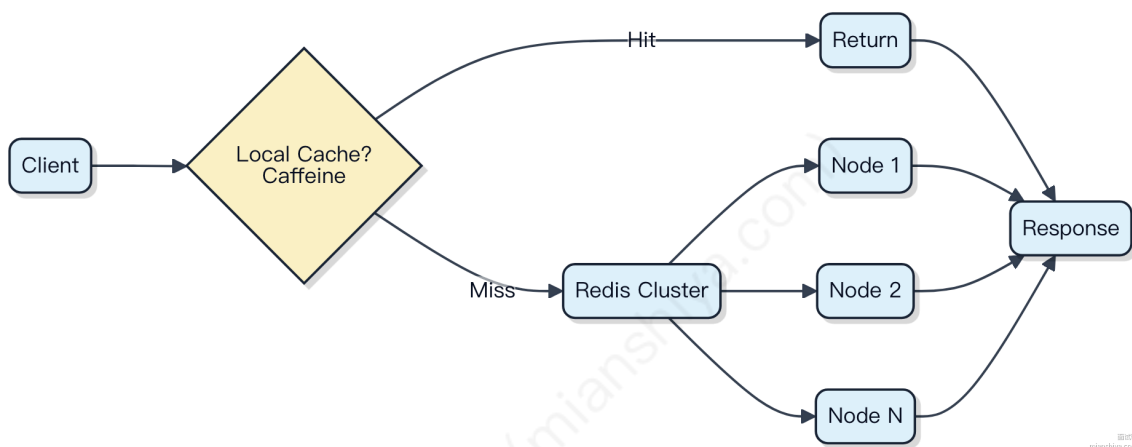
```
GEOADD cities:geo 116.405285 39.904989 "beijing"
GEORADIUS cities:geo 116.4 39.9 100 km WITHDIST
```

整个结构没单独的数据类型，压根不经过特殊编码，就是 zset + geohash 算法组合拳，脏活累活都让 zset 干了。

Redis 性能瓶颈时如何处理？

Redis 出现性能瓶颈，一般先看是 CPU、内存、网络还是磁盘成了短板。大多数情况瓶颈在单线程模型处理大量请求时的吞吐上限。

- 1) 先用 INFO stats 和 redis-cli --latency 观察指标。如果 QPS 上不去但 CPU 没到顶，可能是大 key 或慢查询拖累。像 KEYS 这种阻塞命令得禁用，改用 SCAN。
- 2) 大 key 拆分是常见优化手段。比如一个包含 10 万元素的 hash，读写都慢。可以按业务维度拆成多个小 hash，或者用 Redis Hash 的 slot 机制分散。
- 3) 数据结构选型也很关键。比如用 ZSET 做时间排序队列没问题，但如果频繁修改分数， $O(\log N)$ 的复杂度在百万级数据下就明显了。这时候可以考虑降级为 LIST + 外部索引，或者引入 Redis Streams，它更适合高吞吐的消息场景。
- 4) 真扛不住就横向扩展。Redis Cluster 能自动分片，数据按 key 分布到 16384 个 slot，跨节点访问会重定向。不过 multi-key 操作必须在同一个 slot，可以用 hash tag 强制路由，比如 user:{1000}:profile 和 user:{1000}:orders。
- 5) 极端场景下，本地缓存加一层 Caffeine 或 Guava Cache，把最热的数据挡在 Redis 前面。比如用户登录信息，99% 请求集中在 10% 的 key 上，本地缓存能减少 80% 以上的远程调用。



Redis 中原生批处理命令（MSET、MGET）与 Pipeline 的区别是什么？

原生批处理像把多个命令打包成一个请求发给 Redis，MSET 和 MGET 就是这种。Redis 收到后可以一次性执行，网络来回只有 1 次，指令还是原子的，顺序执行不被打断。

Pipeline 不一样，它不是命令，而是一种通信优化手段。客户端把一堆命令连续发出去，不用等每个响应。Redis 还是一个个处理，但省掉了反复的网络等待时间。比如发 100 条 GET，RTT 是 1ms，传统方式要 100ms，Pipeline 可能只要 2ms（1 次发送 + 1 次回程）。

关键区别在于 **原子性** 和 **实现层级**。MSET/MGET 是服务端支持的单命令多操作，天然原子。Pipeline 是客户端攒请求、批量发，服务端压根不经过“原子执行”这层逻辑，中间可能穿插别的客户端命令。

举个例子：用 Jedis 做 Pipeline，你写：

```
Pipeline p = jedis.pipelined();
p.get("k1");
p.get("k2");
List<Object> resp = p.syncAndReturnAll(); // 批量获取结果
```

这期间其他客户端照样能写数据，没法保证这一组读的原子视图。

再比如，MSET 最多支持 1024 个键，超了得拆；Pipeline 理论上可以塞几千条不同命令，但得自己控制缓冲区大小，别把内存搞爆。

所以，要原子性批量读写小数据，用 MGET/MSET。要是追求吞吐，操作之间不要求原子，比如缓存预热加载 1w 条数据，Pipeline 更合适。

Redis 主从复制的常见拓扑结构有哪些？

Redis 主从复制的拓扑结构其实就那么几种常见模式，不同的场景下选择合适的结构能更好地扛住流量和保证可用性。

1) 一主一从或者一主多从

这是最简单的部署方式。主节点负责写，从节点负责读，读写分离可以分担主库的压力。比如在电商大促时，用多个从库来应对爆发的查询请求。但主节点要是挂了，整个集群就写不了了，得靠 Redis Sentinel 或者手动切换。

2) 树状主从（级联复制）

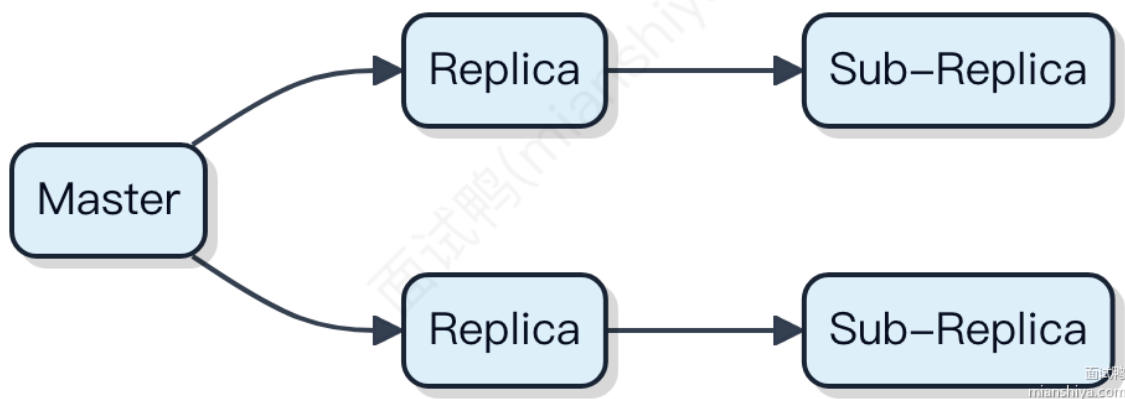
从节点下面再挂更下游的从节点，形成树形结构。这种适合跨机房部署，比如北京主节点同步到上海的中继节点，上

再往下同步给多个本地从库，减少跨机房带宽压力。不过层级多了，数据延迟会逐层累积。

3) 主-从-从（中间节点既是 slave 也是 master）

中间节点开启 `replica-serve-stale-data` 并允许被其他节点复制。它从上游同步数据，同时给下游提供同步服务。这种结构在大规模读场景里很实用，比如用 Redis 做缓存集群，前端业务直接连末端从库。

要注意的是，所有这些结构里，**主节点**始终是唯一的写入点，复制都是单向的。故障转移需要外部干预，除非你搭了 Sentinel 或者 Cluster 模式。



图示：树状主从结构，浅色为中继节点，深绿为末端从库。

Redis List 类型的常见操作命令有哪些？

Redis 的 List 类型底层用双端链表或压缩列表实现，适合做消息队列、最新消息排行这类功能。

1) 插入类命令

往列表头部或尾部塞数据。 `LPUSH key value [value ...]` 从左边加元素， `RPUSH key value [value ...]` 从右边加。比如用 `LPUSH news:latest "breaking"` 把突发新闻推到列表头。

2) 弹出和读取类命令

拿数据但不删用 `LINDEX key index`，按位置查。想删并返回就用 `LPOP key` 或 `RPOP key`，分别从左右两端取出。常配合 `LPUSH + RPOP` 做先进先出队列。

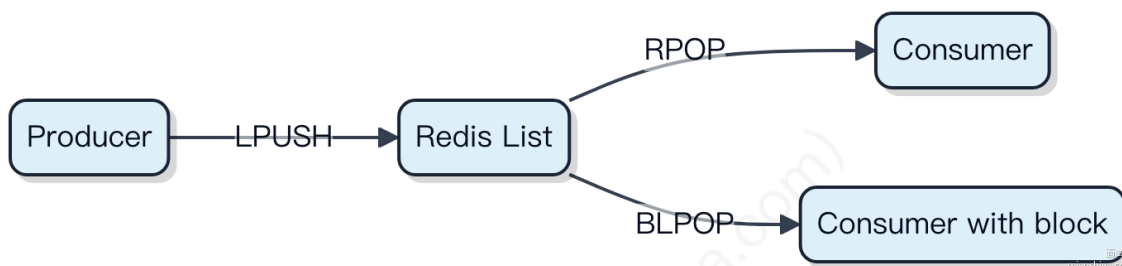
3) 阻塞操作

怕消费者空转可以加个等待时间。 `BLPOP key [key ...] timeout` 在指定 key 上阻塞式取值，超时自动返回，避免客户端频繁轮询。

4) 范围与长度控制

看一段内容用 `LRANGE key start stop`，比如 `LRANGE news:latest 0 9` 拿前十条。设最大长度可以用 `LTRIM key start stop`，防止列表无限膨胀。

有个坑得注意：虽然支持随机访问，但越靠后性能越差，毕竟底层不是数组。高频随机读的场景别硬扛，换 Sorted Set 更合适。



Redis 中如何保证缓存与数据库的数据一致性？

缓存与数据库的一致性问题，本质上是在写操作发生时，如何让缓存和 DB 的状态保持同步。现实中没有完美方案，只有权衡。

1) 先更新 DB，再删除缓存（推荐）

这是 **Cache Aside Pattern** 的标准做法。写请求来临时，先更新数据库，然后把缓存删掉。下次读请求会触发一次缓存重建。这种策略下，缓存不一致的时间窗口很短，且在高并发场景下相对安全。比如用户信息更新，先改 MySQL，再删 Redis 里的 user:1001。

2) 先删缓存，再更新 DB（有风险）

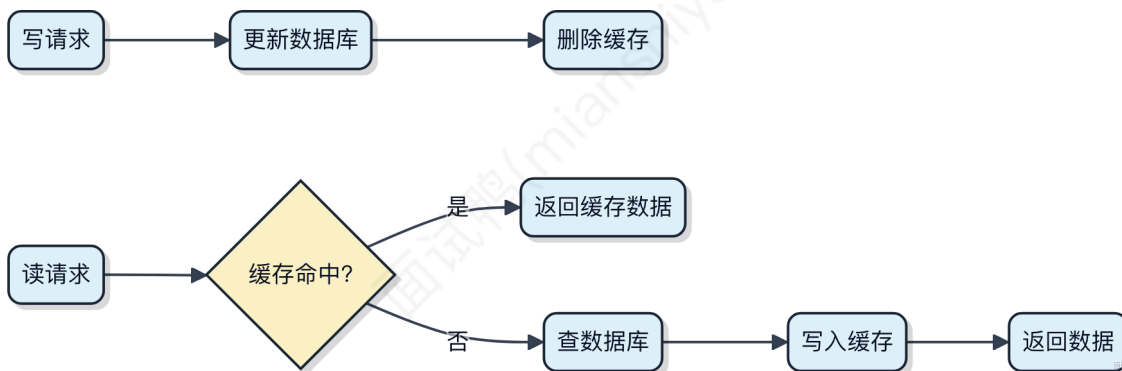
如果删完缓存后，还没来得及更新 DB，这时一个读请求进来，就会把旧数据从 DB 加载到缓存，导致脏数据。这种情况在高并发下可能持续几毫秒，但确实存在。

3) 延迟双删（缓解方案）

针对上面的问题，可以在更新 DB 前先删一次缓存，更新完 DB 后再 sleep 几百毫秒，再次删除缓存。虽然能降低脏数据概率，但 sleep 时间难控制，一般只在强一致性要求高且数据变更频繁的场景用，比如库存系统。

4) 通过 Binlog 异步处理（解耦）

像阿里开源的 **Canal** 可以监听 MySQL 的 Binlog，把数据变更事件发到 MQ，由消费者去清理或更新缓存。这种方式完全解耦，适合复杂系统，比如订单状态变更同步到缓存。



最终建议：优先用“先更新 DB，再删缓存”，配合缓存过期时间兜底。真要强一致，就得上分布式锁或者版本号机制，但性能代价大，多数业务没必要。

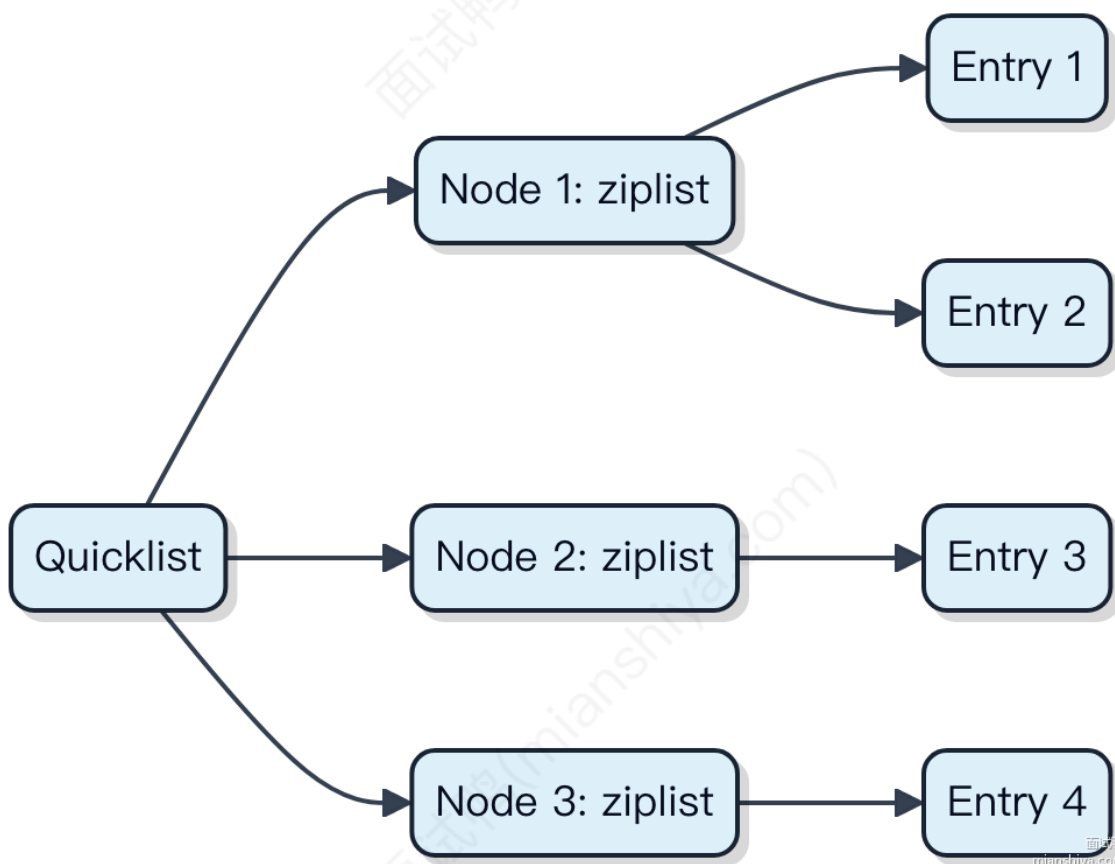
Redis 中的 Ziplist 和 Quicklist 数据结构的特点是什么？

Ziplist 是 Redis 早期用来存储小对象的一种紧凑内存结构，它把数据连续存储在一块内存里，没有指针节点开销。这种设计省空间，但插入删除要移动元素，平均 $O(n)$ 时间成本。当 list 或 hash 的元素少、值小的时候，Redis 会用 ziplist 来编码，比如 `list-max-ziplist-size` 控制节点数或大小。

后来发现 ziplist 扩容缩容容易触发连锁更新，性能不稳定，尤其在大列表场景下压根不经过修改就可能引发内存重排。所以 Redis 3.2 开始用 **Quicklist** 替代 ziplist 实现 list 类型。

Quicklist 本质是个双向链表，每个节点存的是 ziplist。相当于把一个大的 ziplist 拆成多个小段，每段独立维护。这样既能保留 ziplist 内存紧凑的优点，又把修改操作控制在局部，避免整体搬移。你可以通过 `list-max-ziplist-size` 调整每个节点的 ziplist 大小，还能用 `list-compress-depth` 对两端节点做 LZF 压缩，节省内存。

- 1) Quicklist 减少了单次操作的影响范围，插入删除只影响当前 ziplist 片段
- 2) 支持压缩，冷数据可以进一步降低内存占用
- 3) 相比纯 ziplist，稳定性更好，适合生产环境长期运行



现在 hash、zset 等类型的小对象也倾向用 ziplist 编码，但 list 类型已经统一走 Quicklist 了。

Redis 复制延迟的常见原因有哪些？

Redis 复制延迟其实是主从架构里挺常见的问题，一旦出现可能影响数据一致性和故障切换。排查的时候一般从这几个层面来看。

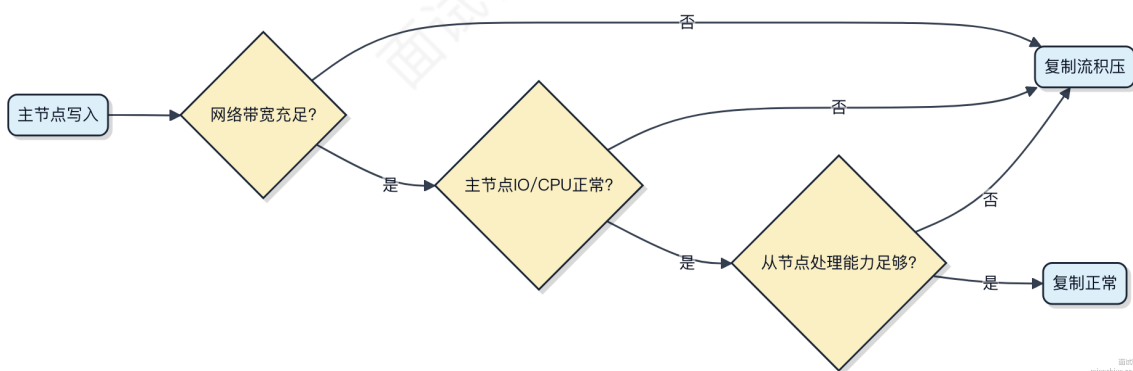
网络带宽跑满是最直接的原因，主节点写入量大时，RDB 快照或命令流同步会占用大量带宽，从节点拉取数据就跟不上了。比如主库每秒写 10MB，但跨机房的链路只有 20Mbps，压根不经过计算就肯定积压。

主节点 CPU 或磁盘压力大也会拖慢复制。像执行 `SAVE` 或后台 `bgsave` 生成 RDB 时，子进程和主进程争抢 CPU 和磁盘 IO，导致主线程处理客户端请求和发送复制流变慢。同样，AOF 日志刷盘策略设成 `everysec` 或 `always`，磁盘写不过来也会影响整体响应。

从节点本身性能不足同样会成为瓶颈。比如它的 CPU 不够强，解析 RDB 或处理命令流的速度跟不上主节点生成的速度，或者内存不足触发 swap，访问速度断崖式下跌。

还有就是单线程复制的问题。Redis 主从复制是单主从链路，即使主节点能发得快，从节点处理慢就会形成堆积。如果拓扑是级联的，中间节点延迟会逐层放大。Kafka 的消费者组能并行消费，但 Redis 从节点只能单线程 replay 命令，这个**阻塞模型**是根本限制之一。

- 1) 检查主节点写入 QPS 和数据变更量
- 2) 观察网络监控是否达到上限
- 3) 排查主从节点的 CPU、内存、磁盘 IO 使用率
- 4) 查看 Redis 日志是否有 `Sync failed` 或 `repl backlog` 相关警告



Redis 事务与关系型数据库事务的主要区别是什么？

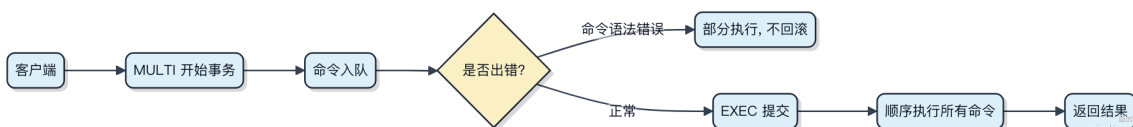
Redis 的事务和 MySQL 这类关系型数据库的事务，看着名字差不多，其实差别挺大的。最根本的一点是，**Redis 不支持回滚**。

你把几个命令塞进 `MULTI` 和 `EXEC` 之间，Redis 会按顺序把它们排队，等 `EXEC` 一发，就一股脑执行下去。哪怕中间某个命令出错了，比如对一个字符串类型执行 `HGETALL`，前面已经执行的命令也不会撤掉，后面的命令照样继续跑。这跟 MySQL 里 `BEGIN` 和 `COMMIT` 包住的操作完全不同，MySQL 出错可以 `ROLLBACK`，所有变更都能撤销。

另一个关键区别是隔离性。Redis 是单线程处理命令的，所以事务一旦开始执行，就不存在并发干扰的问题。它靠这个实现了“串行化”调度，压根不经过锁机制。而 MySQL 要靠 MVCC 或锁来保证不同事务之间的隔离级别。

再就是原子性理解不一样。Redis 的原子性指的是事务里的命令要么都排队成功、要么都不执行（比如 `EXEC` 前断了），但不保证每个命令逻辑上正确。MySQL 的原子性是在存储引擎层做的，配合日志机制确保数据可恢复。

如果你需要类似“回滚”的能力，得靠 `WATCH` 监视键的变化，在提交前检测有没有被其他客户端改过，有变动就放弃执行。这其实是乐观锁的思路，像用 Redis 做库存扣减时就得这么搞。



简单说，Redis 事务更像是“打包执行+有限一致性保障”，脏活累活都靠应用层兜底。

Redis Cluster 模式与 Sentinel 模式的区别是什么？

Redis 的高可用方案里，Cluster 和 Sentinel 是两种常见模式，解决的问题有重叠但定位完全不同。

Sentinel 本质是做**高可用监控与故障转移**。它不负责数据分片，所有主从节点存的是一份数据。当主节点挂了，Sentinel 集群会选举出一个新主，客户端通过监听通知切换地址。整个过程对应用有一定感知，比如连接闪断、短暂只读。典型部署是 3 个 Sentinel 节点 + 1 主 1 从或更多从，像 Codis 就依赖这种架构做代理层后端。

Cluster 则是原生的分布式方案，核心是**数据分片和去中心化**。16384 个哈希槽把数据分散到多个主节点，每个主可以带从节点做高可用。请求先根据 key 计算 slot，再路由到对应节点。客户端要么直连，要么通过 smart 客户端自动重定向。扩容缩容靠迁移 slot 实现，比如 RedisShake 可辅助迁移。

两者最大区别在于是否分片。Sentinel 模式下数据全量复制，容量受限于单机；Cluster 能横向扩展，扛住几十 GB 到 TB 级数据。但 Cluster 运维复杂，跨 slot 的事务和批量操作受限；Sentinel 架构简单，适合中小规模场景。

选型上，数据量大、需要水平扩展直接上 Cluster；追求稳定简单，数据量不大，用 Sentinel 搭配客户端或代理更省心。

Redis 的 ListPack 数据结构是什么？

从 Redis 7.0 开始，ListPack 成为了 **Hash** 和 **ZSet** 内部结构的默认编码之一，用来替代逐渐暴露短板的 ZipList。

它本质是一个紧凑的、连续内存存储的序列化结构，把多个小元素挨着存，省去了指针开销。每个元素直接存自己的内容和长度，读的时候按格式一步步解析出来。这种设计让内存利用率特别高，适合存短字符串或整数这类小数据。

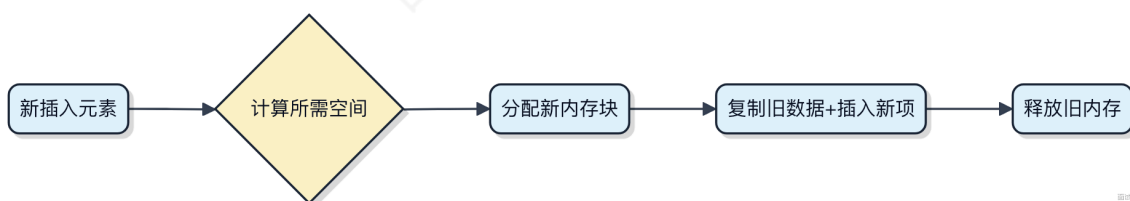
相比老将 ZipList，ListPack 做了关键改进。ZipList 在修改中间元素时可能触发连锁更新，一个改动态长字段的操作可能波及后面所有节点，性能不可控。而 ListPack 每个元素头里记录的是自己占用的总字节长度，修改只影响当前项，不会扩散，稳定性强得多。

不过它也有硬伤，就是所有操作都是 $O(n)$ 的。查第 5 个元素？只能从头开始一个个跳。所以官方用它也是有前提的——只在元素个数少于 512 个，且每个值大小不超过 64 字节时启用，超出就切到 Hash Table 或 SkipList。

代码层面你看不到直接操作 ListPack 的 API，它是完全内部的。但你可以通过配置控制切换阈值：

```
listpack-max-listpack-size 512
listpack-max-listpack-entries 64
```

实际场景中，像用 ZSet 做滑动窗口限流，或者 Hash 存用户会话的小属性，只要数据够小，Redis 就会自动用 ListPack 编码，内存能省下 30% 以上。



整个结构每次变更都要重新分配内存，所以不适合频繁修改的大列表。它的定位很明确：小数据、高密度、低频修改的冷热数据混合场景下的内存优化利器。

Redis 中的内存碎片化是什么？如何进行优化？

内存碎片化指的是 Redis 分配的内存中，存在大量不连续的小块空闲内存，导致无法有效利用。即使总空闲内存足够，也可能因为没有连续的大块内存而无法分配给新数据。

Redis 使用 jemalloc 作为默认内存分配器，它会按不同大小的块进行内存管理。随着数据频繁增删改，小块内存被释放后可能散布各处，形成碎片。**内存碎片率**通过 `mem_fragmentation_ratio` 指标体现，计算方式是：

```
used_memory_rss / used_memory
```

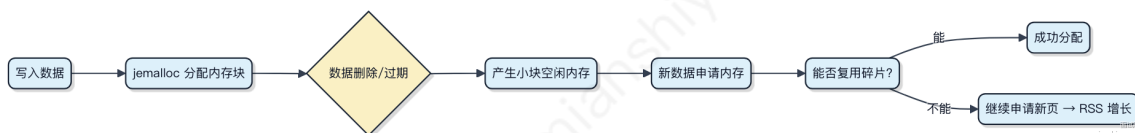
- 接近 1.0：理想状态
- 大于 1.5：可能存在严重碎片
- 小于 1.0：说明部分内存被交换到磁盘，性能已受损

优化手段主要有三种：

- 1) 启用 **内存碎片自动回收**。从 Redis 4.0 开始支持 `activedefrag yes` 配置，配合 `active-defrag-ignore-bytes` 和 `active-defrag-threshold-lower` 等参数，在运行时逐步整理碎片，但会消耗额外 CPU。
- 2) 应用层控制键值大小一致性。避免频繁创建和删除大对象，尤其是几 KB 到几十 KB 的中间尺寸对象，这类最易造成 jemalloc 分配失衡。
- 3) 极端情况下采用重启重建。通过 RDB 或 AOF 文件恢复数据，重新加载时内存布局更紧凑。比如使用主从切换或集群迁移实现无感重启。

代码层面注意批量操作，减少频繁小写入：

```
// 批量插入比单条循环高效得多
for (int i = 0; i < keys.length; i++) {
    pipeline.set(keys[i], values[i]);
}
pipeline.sync();
```



在 Redis 集群中，如何根据键定位到对应的节点？

Redis 集群通过 **一致性哈希的变种算法**来定位键所在的节点，但不是传统的一致性哈希，而是用了“哈希槽”（hash slot）机制。

- 1) Redis 集群总共定义了 16384 个哈希槽。每个键通过 CRC16 算出一个 0 到 65535 的哈希值，再对 16384 取模，确定属于哪个槽。
- 2) 每个主节点负责一部分哈希槽。比如有 3 个主节点，槽可能是这样分配：节点 A 负责 05500~5500，B 负责 5500~11000，C 负责 11001~16383。
- 3) 定位过程是客户端直接计算的。客户端拿到 key 后，算出 hash slot，然后查本地缓存的 slot -> node 映射表，就知道该发给哪个节点。

如果映射关系变了，比如扩容或缩容，节点会返回 **MOVED** 重定向，告诉客户端新位置。客户端更新本地映射后，后续请求就直接走新路由。

这种设计让集群具备去中心化特性，路由逻辑压根不经过中间代理，全靠客户端+重定向同步元数据。

要注意的是，涉及多个 key 的操作必须在同一个 slot，否则会报错。可以用哈希标签强制多个 key 落在同一 slot，比如 `{user1000}.profile` 和 `{user1000}.orders`。

说说 Redisson 分布式锁的原理？

Redisson 的分布式锁核心是基于 Redis 的 **Lua** 脚本和 Redis 单线程特性来保证原子性，同时支持可重入、锁续期等高级功能。

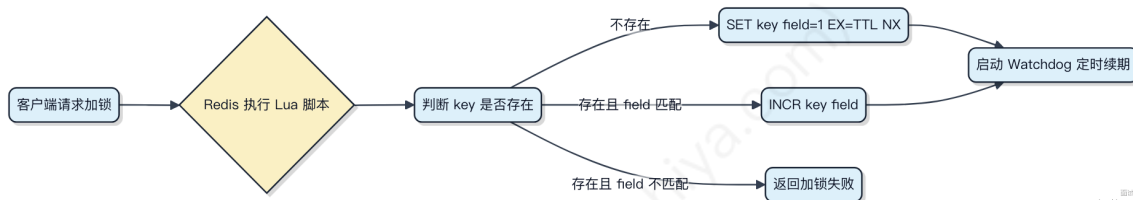
客户端加锁时，会通过一段 Lua 脚本向 Redis 写入一个 hash 结构，key 是锁名，field 是当前线程的唯一标识（比如 `UUID + threadId`），value 是重入次数。如果 key 不存在，就设置过期时间防止死锁；如果已存在且 field 匹配，就将 value 加一，实现可重入。

解锁同样是 Lua 脚本操作，先校验 threadId 是否匹配，不匹配直接返回失败；匹配则将重入数减一，减到 0 就删除 key 并释放锁。

最关键的是**锁续期机制**，也就是看门狗（Watchdog）。当客户端拿到锁后，会启动一个后台任务，每过 1/3 过期时间就检查锁是否还被持有，如果还在用就自动延长 TTL，避免业务还没执行完锁就过期了。

这种设计让客户端不需要手动管理超时，只要线程活着就能保住锁。像 `RLock rlock = redisson.getLock("order"); rlock.lock();` 这样简单调用背后其实已经包含了网络通信、脚本执行和定时任务调度这些脏活累活。

使用场景上，适合订单扣减、库存超卖这类强一致性要求的场景，但要注意网络分区下可能丢失锁的问题，极端场景建议结合 RedLock 模式，不过一般单 Redis 实例配合合理超时也能扛住大部分情况。



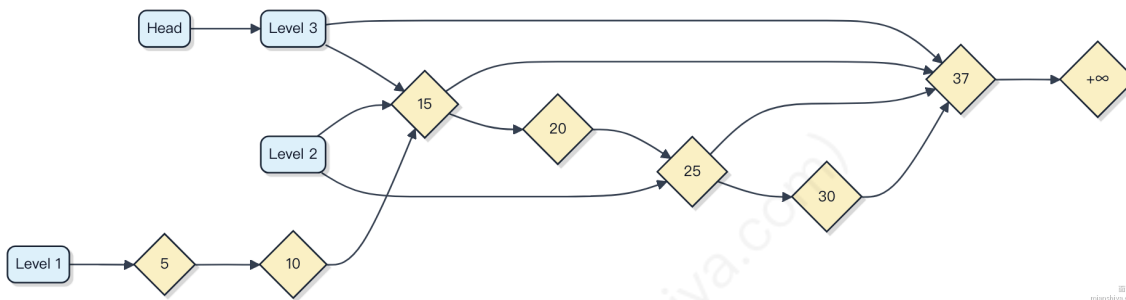
为什么 Redis Zset 用跳表实现而不是红黑树？B+树？

Redis 的 Zset 选择跳表（Skip List）实现，主要是在 **读写性能、实现复杂度和范围查询** 之间找了个好平衡。

1) 跳表在平均情况下，插入、删除、查找都是 $O(\log n)$ ，虽然最坏情况可能退化一点，但随机层数设计让它大概率保持均衡，不像红黑树那样需要频繁旋转维持平衡。你想想，每次插入都要左旋右旋，写放大严重，对 Redis 这种高并发场景来说，脏活累活越少越好。

2) 范围查询是硬需求，比如 `ZRANGEBYSCORE`。跳表天然支持双向链表+多层索引，指针直接串着，遍历效率高。红黑树做中序遍历也可以，但得靠额外数据结构辅助，代码复杂。B+树倒是擅长范围扫描，但它是为磁盘 IO 设计的，一个节点对应一页，内存里用反而有额外指针开销，没必要。

3) 跳表代码实现简单，调试容易。Redis 强调稳定和可维护，不用搞复杂的树平衡逻辑，开发成本低。而且实测性能不输红黑树，甚至在并发插入时表现更好。



你看这个结构，一层层往下筛，查得快，删增也方便，指针一改就行。Redis 要的就是这种“够快、够稳、别太复杂”的方案。

Redisson 看门狗 (watch dog) 机制了解吗？

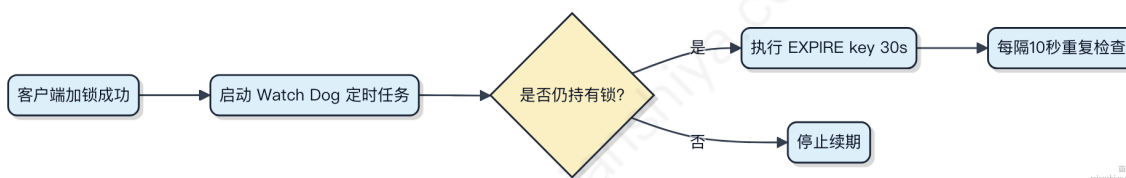
Redisson 的看门狗机制是解决分布式锁自动续期的关键设计，主要针对业务执行时间不确定的场景。

你拿到锁之后，如果业务还没处理完，锁就过期了，那其他线程就能进来，这显然不行。Redisson 会在后台起一个 Watch Dog 线程，默认每 10 秒检查一次，只要线程还持有这个锁，它就会自动把锁的过期时间延长到 30 秒（基于默认的 `lockWatchdogTimeout` 配置）。这个动作完全由客户端自己触发，**压根不经过业务逻辑干预**。

这个机制只在没给锁设置超时时间的时候生效。比如你用的是 `lock()` 而不是 `tryLock(10, TimeUnit.SECONDS)`。一旦指定了超时时间，就靠你自己控制节奏了，看门狗不会介入。

实际使用中，像 `RLock rLock = redisson.getLock("order"); rLock.lock();` 这样调用后，续期就是透明的。但要注意，如果 Redisson 实例宕了，Watch Dog 也跟着挂，所以依赖的是客户端的存活状态。

常见问题是在集群故障转移时可能出现多个客户端认为自己持有锁的情况，这时候就得靠 RedLock 或者更严格的 fencing token 机制来兜底。



如何解决 Redis 中的热点 key 问题？

热点 key 问题说白了就是某个 key 被疯狂访问，QPS 可能上十万，直接把 Redis 实例的 CPU 或网卡打满，甚至拖垮整个服务。

1) 先定位。一般通过 `redis-cli --hotkeys` 或监控系统（比如用 Prometheus + Grafana）看 key 的访问频率。生产环境建议在 proxy 层（比如 TendisProxy 或 Codis）埋点统计 top N 热 key。

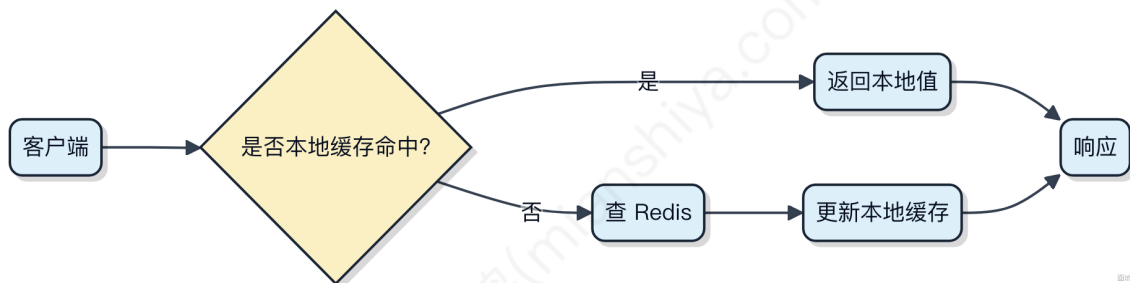
2) 解决思路不是单一的，得组合拳。

本地缓存是第一道防线。应用节点用 Caffeine 或 Guava Cache 把热 key 缓一份，设置短过期时间（比如 1~2 秒），这样大部分请求就压根不经过网络到 Redis。注意缓存穿透问题，可以用布隆过滤器兜底。

3) 拆分 key 也是常用手段。比如原来一个 key 存全量数据，改成 key_1、key_2 分片存储，访问时随机或轮询拉取，把压力分散。适用于静态资源类场景，比如首页 banner。

4) Redis 侧可以做多副本。比如给这个 key 单独起一个只读副本集群，或者利用 Redis Cluster 的 replica 只读节点分流读请求。

5) 极端情况，写扩散。比如把热 key 的值推送到所有客户端内存里，靠消息中间件（比如 Kafka）通知更新。这招重运维，搞不定一致性别轻易上。



Redis 为什么这么快?

Redis 快不是单一原因造就的，是多个设计选择叠加的结果。很多人第一反应是“因为内存”，但这只是基础，真正关键的是它怎么利用这个基础。

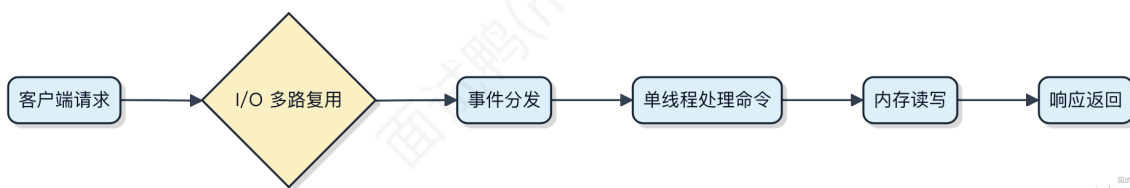
首先，数据存在内存里确实让访问速度上了个台阶，读写都在纳秒到微秒级。但光这样还不够，比如你用 Java 写个 Map 存数据，也快，但扛不住并发请求。Redis 的**多路复用 I/O 模型**才是让它能同时处理几万连接的核心。

它用 epoll (Linux) 这类机制，一个线程就能监听大量 socket 连接，有数据才处理，压根不经过阻塞等待。像 Tomcat 那种每个请求起个线程的方式，在高并发下线程切换开销直接搞垮性能，而 Redis 单线程处理命令，避免了上下文切换和锁竞争，脏活累活都省了。

当然，单线程也有前提：命令执行必须足够快。所以你得注意别用 `KEYS *` 这种慢操作，一执行全库卡住。官方推荐用 `SCAN` 代替。

数据结构上，Redis 直接基于 C 语言实现紧凑结构，比如 SDS 字符串比普通字符串少内存拷贝，哈希表查找接近 $O(1)$ ，这些细节累积起来也让响应更快。

再往后看，像 Pipeline 批量发命令、主从复制异步同步、RDB 快照 fork 子进程由操作系统做写时复制，都是围绕“减少阻塞”做的权衡。



总结一句：**内存 + I/O 多路复用 + 单线程 + 精简数据结构**，这套组合拳打下来，才让 Redis 能轻松扛住 10w+ QPS。

为什么 Redis 设计为单线程？6.0 版本为何引入多线程？

Redis 的单线程模型其实指的是它的核心网络 I/O 和命令执行流程由一个线程串行处理。这么设计最直接的原因是，**避免锁竞争**带来的复杂性和性能损耗。内存操作本身极快，如果命令处理再用多线程加锁，上下文切换和同步开销反而可能压倒收益。

尤其是在 10Gbps 网络下，Redis 处理简单命令的瓶颈通常不在 CPU，而在于网络读写。所以早期版本选择用 **I/O 多路复用 + 单线程事件循环**，像 Redis 这种以微秒级响应为目标的系统，逻辑越简单越可控。

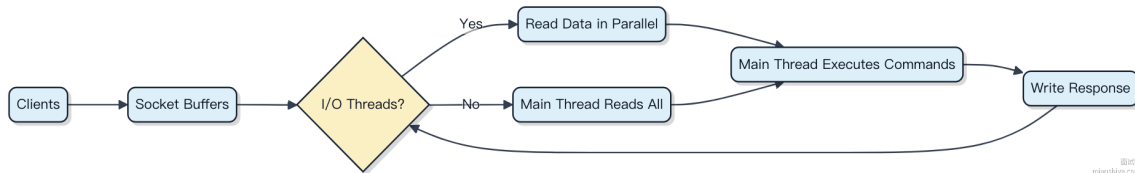
但问题出在高并发场景下，网络 I/O 成了新瓶颈。比如同时有上万连接发来请求，单线程读取 socket 缓冲区就成了串行步骤，CPU 压根没被充分利用。

从 Redis 6.0 开始，它引入了多线程来分担这部分脏活累活。具体做法是：主线程继续处理命令调度和执行，但把网络数据的读写拆给 I/O 线程池。你可以通过 `io-threads` 配置开启，比如设为 4，那么读请求就由 4 个线程并行从 socket 读数据，解析完再交回主线程统一执行。

```
// redis.conf 关键配置
io-threads 4
io-threads-do-reads yes
```

注意，命令执行依然在主线程，保证了原子性。多线程只解决“数据搬入搬出”的问题。实测在多核机器上，开启 4 线程能提升吞吐近一倍。

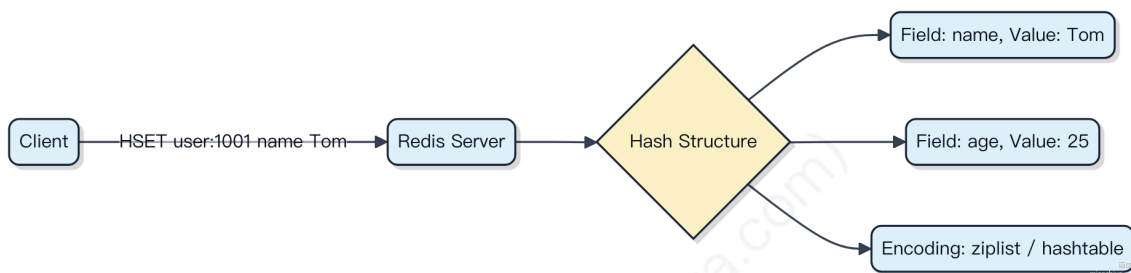
- 1) 单线程为规避锁开销，依赖内存速度和 I/O 多路复用
- 2) 6.0 多线程仅用于网络 I/O，不碰命令执行
- 3) 适合高并发、多核环境，小并发开多线程反而增加切换成本



Redis 的 hash 是什么？

Redis 的 hash 本质上就是一个字段和值的映射表，类似 Java 里的 `Map<String, String>` 或 `Map<String, Object>`，但它是存放在一个 key 下的结构。你可以用它来存储对象，比如用户信息，不用把整个对象序列化成一个字符串。

- 1) 你往一个 hash 里塞数据，用的是 `HSET user:1001 name "Tom" age "25"` 这种命令，读的时候用 `HGET user:1001 name`。这样改某个字段特别方便，不用动整个对象。
- 2) 底层实现上，Redis 会根据数据量自动切换两种编码方式。小的 hash 用 **ziplist**（压缩列表），内存紧凑，所有字段和值挨着存；当字段多了或单个值太大，比如超过 64 字节，或者字段数超过 512 个，就会转成 **hashtable**，也就是真正的哈希表，读写都是 $O(1)$ 。
- 3) 适用场景很明显，比如缓存一个商品信息，字段多但更新独立，用 hash 就很合适。像在电商系统里用 Redis 存商品详情，字段如 price、stock、title 各自更新互不影响。但如果你每次都读整个对象，直接用 string 存 JSON 反而更简单。
- 4) 注意别滥用嵌套，Redis 不支持 hash 套 hash。你要存复杂结构，得自己拼 key，比如用 `user:1001:profile` 和 `user:1001:settings` 分开存。



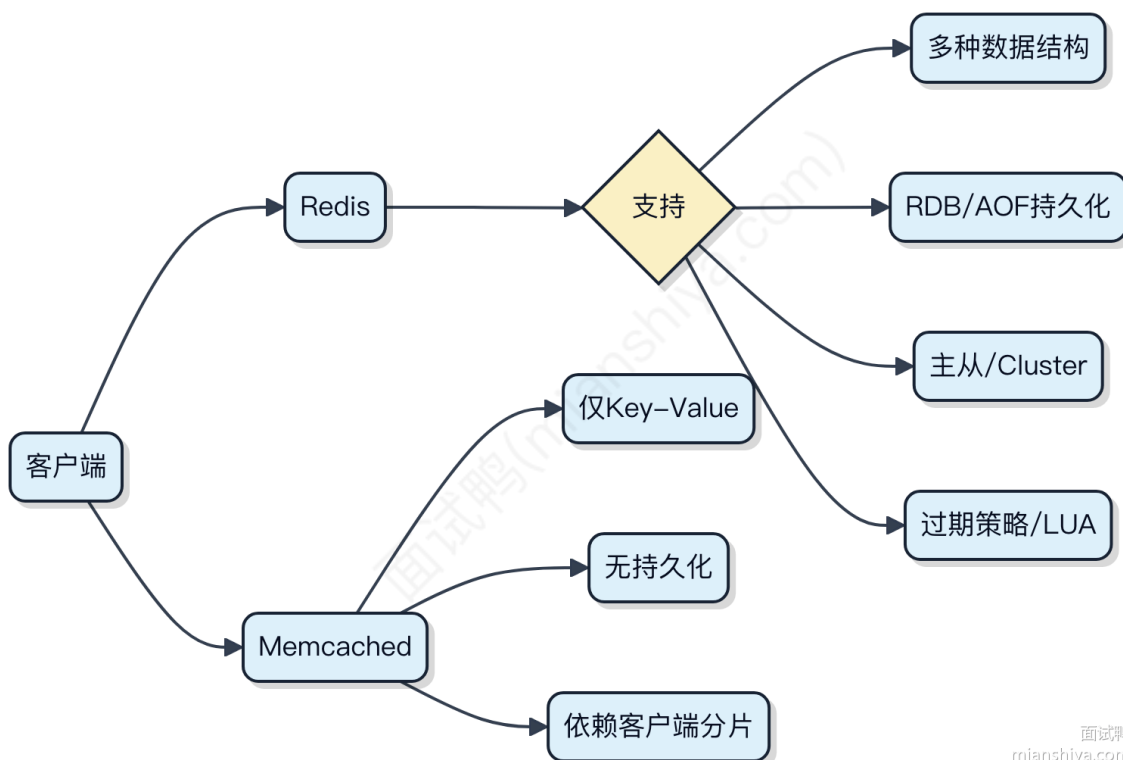
这种设计让 hash 在内存和性能之间做了平衡，小对象省空间，大对象保速度。

Redis 和 Memcached 有哪些区别？

Redis 和 Memcached 都是内存数据存储系统，但设计目标和能力差异明显。

- 1) 数据类型支持。Memcached 只支持简单的 key-value 字符串，存什么取什么。Redis 支持 **String**、Hash、List、Set、Sorted Set、Stream 等丰富类型，能直接操作数据结构，比如用 **HINCRBY** 原子更新哈希字段。
- 2) 持久化能力。Memcached 数据全在内存，重启就丢。Redis 提供 RDB 快照和 AOF 日志两种机制，可以落盘，故障后能恢复，适合对数据可靠性有要求的场景，比如用户会话存储。
- 3) 高可用与集群。Memcached 本身不带主从复制，靠客户端分片。Redis 支持主从同步、哨兵模式自动 failover，以及 Redis Cluster 实现分布式部署，支撑更大规模读写。
- 4) 线程模型。Memcached 用多线程处理并发，多核利用率高。Redis 6.0 之前是单线程事件循环，避免锁竞争，6.0+ 引入了多线程 I/O，但命令执行仍是单线程，保证原子性。
- 5) 内存管理。两者都用预分配内存池减少碎片，但 Redis 对内存更精细控制，支持 LRU 近似淘汰策略，还能设置 key 的 TTL 自动过期。

一般选型时，如果只是做纯缓存，追求极致并发和简单，Memcached 在某些场景仍有优势。但大多数现代应用倾向于 Redis，因为它功能更全面，能当缓存也能当轻量数据库用，像 Session 存储、排行榜、消息队列都能搞定。



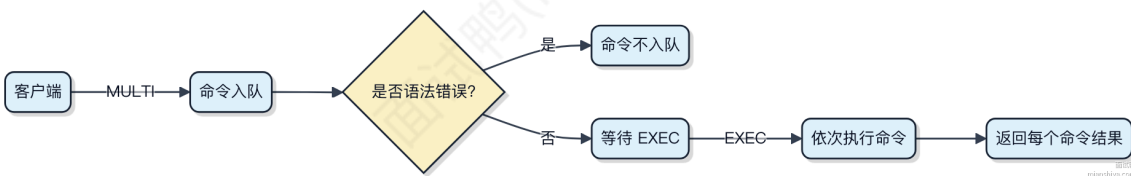
面试鸭
mianshiya.com

Redis 支持事务吗？如何实现？

Redis 确实提供了事务支持，但和传统数据库的事务不一样，它不保证原子性，也不支持回滚。你把一组命令用 `MULTI` 包起来，Redis 会把这些命令放到队列里，然后通过 `EXEC` 一次性发出去，按顺序执行。

1) 开启事务用 `MULTI`，之后的命令不会立即执行，而是进入等待状态。2) 命令被缓存到客户端对应的队列中，直到调用 `EXEC` 或 `DISCARD`。3) 执行 `EXEC` 后，Redis 按顺序串行执行队列里的命令，期间不会被其他客户端中断。

有个关键点，如果某个命令执行出错，比如对字符串类型执行 `HSET`，Redis 不会回滚已经执行的命令，错误命令后面的命令照样继续执行。这也是和 MySQL 这类数据库最大的区别。



面试鸭
mianshiya.com

可以用 Lua 脚本来弥补这个缺陷，因为 Redis 执行 Lua 脚本是原子的，要么全执行，要么全不执行，能实现真正的原子性操作。像 `redisson` 这类 Java 客户端就是靠 Lua 脚本实现分布式锁的。

另外，Redis 事务也不隔离，中间不会加锁，多个事务可能交错读写同一个 key，需要你自己控制并发访问。

Redis 数据过期后的删除策略是什么？

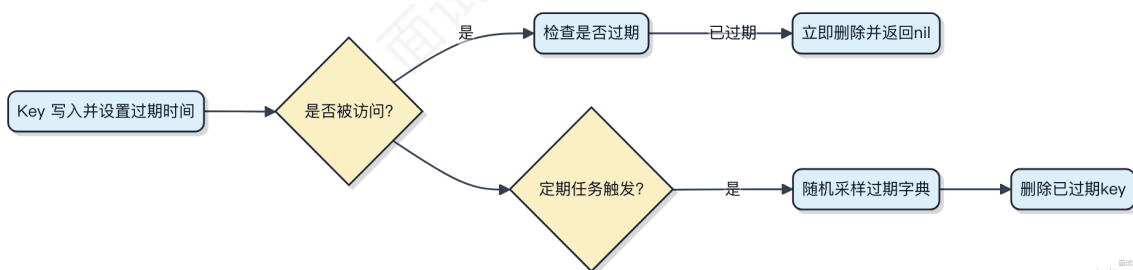
Redis 的过期键删除采用的是 **惰性删除** 和 **定期删除** 两种策略组合，既保证内存能被及时释放，又避免消耗太多 CPU 资源。

1) 惰性删除是在访问 key 的时候才检查它是否已过期，过期就删掉。比如执行 GET 操作时，Redis 会先查一下这个 key 是否存在且已过期，如果过期就删除并返回 nil。这种方式简单直接，但有个问题：如果一个 key 过期了却没人访问，那它就会一直占着内存。

2) 为了解决这个问题，Redis 还启用了定期删除。它会周期性地随机抽查一部分带过期时间的 key，删掉其中已经过期的。默认每秒运行 10 次，每次最多处理 25 个 key（具体数量可配），并且只会扫描设置了 TTL 的字典。这样可以在不影响性能的前提下，主动清理掉一批“僵尸 key”。

这两种机制互补：定期删除负责兜底，把大部分过期 key 清理掉；惰性删除则确保任何过期 key 都不会在被访问时还活着。

实际场景中，像用 Redis 做缓存、会话存储（比如 Spring Session + Redis）都依赖这套机制。如果你发现内存增长异常，可能就是因为过期 key 没被及时删，得看看是不是写得多读得少，导致惰性删除没触发。



Redis 中有哪些内存淘汰策略?

Redis 内存淘汰发生在物理内存不够时，会根据配置的策略清理一些 key 来腾出空间。你得知道，这直接关系到缓存命中率和数据可靠性。

1) noeviction

默认策略，内存满了就不再接受写操作，读请求还能正常处理。适合不能容忍数据丢失的场景，比如用作存储而不是缓存。

2) volatile-lru

只对设置了过期时间的 key 执行 LRU（最近最少使用）淘汰。像 Session 缓存这种有明确生命周期的数据就很合适，避免影响永久数据。

3) allkeys-lru

对所有 key 做 LRU 淘汰，不管有没有设置 TTL。这是最常用的策略之一，适合纯缓存场景，比如用 Redis 做热点数据缓存，像电商商品信息这类。

4) volatile-lfu 和 allkeys-lfu

LFU 是基于访问频率淘汰，比 LRU 更能应对突发流量。allkeys-lfu 现在是推荐的默认策略，尤其适合访问分布不均的场景，比如社交 feed 流。

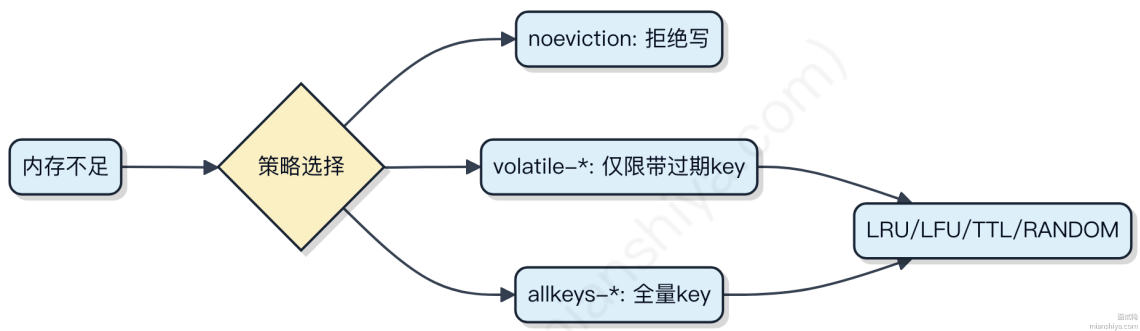
5) volatile-ttl

优先淘汰剩余生存时间短的 key，适用于大量短期 key 的堆积场景。

6) volatile-random 和 allkeys-random

随机删除，简单粗暴。性能开销小，但命中率可能差，一般不作为首选。

实际项目中，如果你用 Redis 做热点缓存，**allkeys-lfu** 或 **allkeys-lru** 能更好地扛住流量。如果混着存了关键数据，就得小心别误删，可以考虑 **noeviction** 配合监控告警。



选哪个，得看你数据的访问模式和能不能容忍丢失。

Redis 的 Lua 脚本功能是什么？如何使用？

Redis 的 Lua 脚本让你能把一组操作打包成一个原子执行的脚本，整个过程不会被其他命令打断。这在需要保证多个 key 操作一致性时特别有用，比如分布式锁、库存扣减这种场景。

Lua 脚本通过 **EVAL** 或 **EVALSHA** 命令执行。**EVAL** 直接传脚本内容，**EVALSHA** 用脚本的 SHA1 值调用，避免重复传输代码，提升性能。

脚本里可以用 **KEYS** 和 **ARGV** 传参，Redis 会把这两个表映射成客户端传入的键和值。一般建议把会被频繁调用的脚本先加载到 Redis 中，用 **SCRIPT LOAD** 预加载，再用 **EVALSHA** 触发。

1) 写个简单的库存扣减脚本：

```
local num = redis.call('GET', KEYS[1])
if not num then return 0 end
if tonumber(num) >= tonumber(ARGV[1]) then
    return redis.call('DECRBY', KEYS[1], ARGV[1])
else
    return -1
end
```

2) 用 EVAL 执行：

```
EVAL "上述脚本" 1 stock_key 10
```

注意点：脚本得尽量快，别做复杂循环或耗时操作，否则会阻塞主线程。单个脚本执行时间最好控制在几毫秒内，不然会影响其他请求响应。

像 **原子性**、**减少网络开销**、**逻辑复用** 是它的主要优势。但搞不定长时间任务，真要处理复杂业务逻辑，还是得靠服务端程序。



Redis 的 Pipeline 功能是什么？

Redis 的 Pipeline 是一种批量执行命令的机制，用来减少客户端与服务端之间的网络往返次数。每次 Redis 命令都要经历“发送命令 → 等待响应”的过程，如果连续发 100 条命令，就会有 100 次 RTT（往返时延）。Pipeline 能把多条命令一次性打包发送，服务端依次处理后批量返回结果，整个过程**压根不经过逐条等待**。

比如你要 set 1000 个 key，不用 Pipeline 至少要 1000 次网络交互；用了之后，可能只要 1 次或几次就能完成，性能提升非常明显，在千兆网络下能从几百 QPS 提升到几万 QPS。

它和事务不一样，Pipeline 不保证原子性，也不支持回滚，纯粹是优化网络开销的手段。MSET、HSET 这类原生批量命令当然更好，但如果你要混合执行不同类型的命令，比如 set、del、incr 各几十个，Pipeline 就是最佳选择。

代码上也很简单，以 Jedis 为例：

```
try (Jedis jedis = new Jedis("localhost")) {
    Pipeline pipeline = jedis.pipelined();
    for (int i = 0; i < 1000; i++) {
        pipeline.set("key:" + i, "value" + i);
        pipeline.incr("counter");
    }
    List<Object> results = pipeline.syncAndReturnAll();
}
```

- 1) 命令通过 pipelined() 获取管道对象后，调用不会立即执行
- 2) 实际是把命令缓存在本地队列里
- 3) syncAndReturnAll() 才真正发送并等待所有回复

Redis 中的 Big Key 问题是什么？如何解决？

Big Key 指的是 Redis 中单个 key 对应的 value 过大，比如一个 String 超过 10MB，或者一个 Hash、ZSet 成员数量超过几万个。这种 key 会带来一系列连锁问题。

- 1) 网络传输上，一次读取或写入 Big Key 可能占满带宽，导致其他请求被阻塞。
- 2) 在主从复制时，RDB 或 AOF 重放阶段，Big Key 的加载会拖慢整个恢复过程。
- 3) 内存分配和释放时，Redis 是单线程处理命令，Big Key 的删除操作可能引发长时间阻塞，甚至触发 DEL 命令延迟告警。

常见的解决方案有几个方向。第一是**拆分**，把一个大结构拆成多个小 key。比如原来用一个 Hash 存百万用户积分，现在按用户 ID 分片，用 user:score:001、user:score:002 这样分散存储。读写时通过计算定位具体 key。

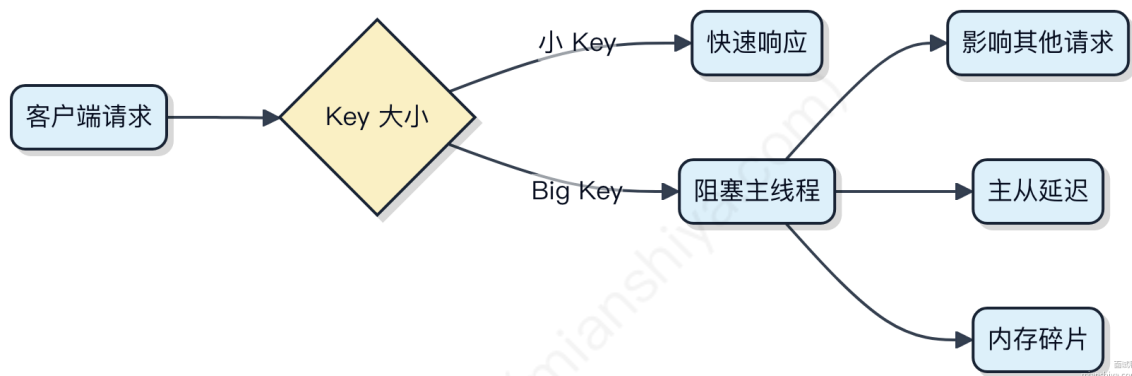
第二是**压缩数据结构**。如果 value 是文本或 JSON，可以考虑用 gzip 压缩后再存入，取出来再解压。虽然增加 CPU 开销，但能显著降低内存和网络开销。

第三是**使用合适的数据类型**。比如本来用 String 存序列化的大对象，不如拆成多个 field 用 Hash，这样可以部分更新，避免全量读写。

代码层面注意批量操作的粒度：

```
// 避免一次性 hgetall 一个超大 hash
List<String> fields = getFields(); // 控制一次只取少量 field
Map<String, String> part = jedis.hgetAll("user:profile");
```

监控上可以用 `redis-cli --bigkeys` 定期扫描，结合 Redis 自身的 slow log 和内存分析工具发现隐患。



Redis 中的缓存击穿、缓存穿透和缓存雪崩是什么？

缓存击穿是指某个热点 key 在失效的瞬间，大量请求直接打到数据库。因为缓存没命中，数据库扛不住瞬时流量，比如商品秒杀场景下库存 key 过期，成千上万请求同时查库。

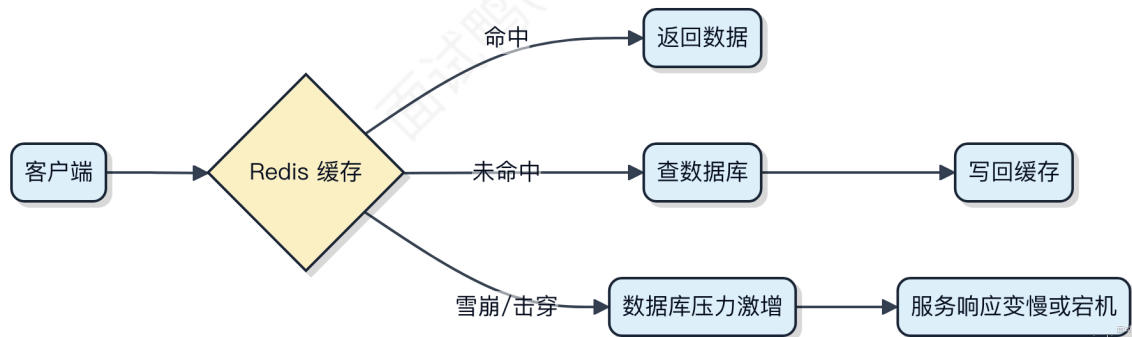
- 1) 可以通过**逻辑过期**或永不过期策略避免物理失效。比如把过期时间存在 value 里，应用层判断是否逻辑过期，再异步更新。
- 2) 也可以用互斥锁，比如 Redis 的 `SETNX` 控制只有一个线程去加载数据，其他线程等待。但要注意锁超时和单点故障问题，结合 Redlock 谨慎使用。

缓存穿透是查询一个根本不存在的数，缓存和数据库都没有，导致每次请求都落库。恶意攻击或非法 id 都可能触发。

布隆过滤器是常见解法，提前拦截无效请求。请求先过布隆过滤器，存在才查缓存和数据库。Guava 或 Redisson 都有实现，注意误判率和内存占用平衡。

缓存雪崩是大量缓存同时失效，大量请求击穿到数据库。比如缓存集群宕机，或者一批 key 设置了相同的过期时间。

解决方案一是过期时间加随机值，比如基础时间 + 300 秒内随机偏移，避免集体退役。二是做二级缓存，比如本地 Caffeine 缓存兜底，虽然可能脏读但能抗住压力。



这三个问题本质都是缓存不可用导致数据库被压垮，解决思路无非是“挡”、“防”、“限”。挡是用布隆过滤器挡住非法请求，防是防止热点失效，限流是最后一道防线，像 Sentinel 可以配合使用。

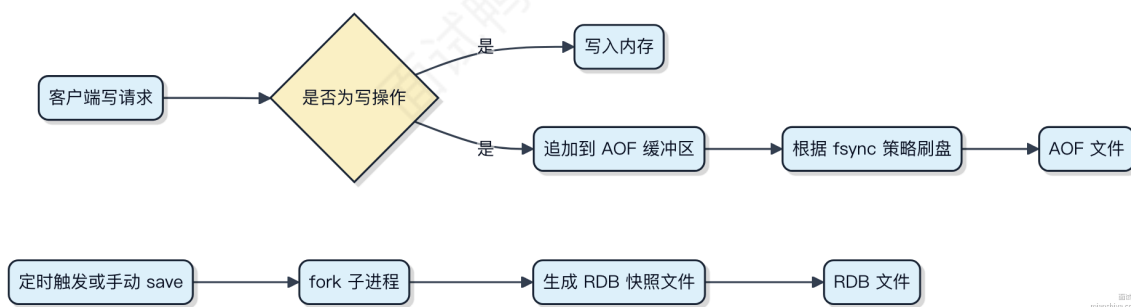
Redis 的持久化机制有哪些？

Redis 的持久化主要是为了在实例重启后能恢复数据，它提供了两种核心机制，实际应用中经常组合使用。

1) RDB 是某个时间点的数据快照。它会 fork 一个子进程，把当前内存数据全量写入 RDB 文件。文件紧凑，恢复速度快，适合做备份。比如用 Redis 做缓存集群时，每天凌晨生成一次 RDB，相当于一次“全量归档”。但两次快照之间如果宕机，中间的数据就丢了，所以无法做到完全不丢数据。

2) AOF 记录的是每一条写操作命令。它像数据库的 binlog，以追加方式写入 aof 文件。通过配置 fsync 策略控制刷新频率。appendfsync everysec 最常用，性能和安全性比较均衡，最多丢 1 秒数据。不过 AOF 文件通常比 RDB 大，恢复也慢一些，因为要重放命令。

生产环境一般同时开启 RDB 和 AOF。Redis 重启时优先使用 AOF 恢复数据，更完整。AOF 重写机制可以定期压缩日志，去掉无效操作，避免文件无限膨胀。



Redis 在生成 RDB 文件时如何处理请求？

Redis 在生成 RDB 文件时用的是 **fork** 子进程，这个设计很关键。父进程继续处理客户端请求，子进程把内存数据快照写到磁盘，两个进程共享同一份内存数据，互不干扰。

1) fork 之后子进程拿到的是那一刻的内存副本，所以后续的写操作不会影响正在生成的 RDB 内容，保证了快照的一致性。

2) 所有读请求正常由父进程处理，写请求也照样能接，Redis 不会因为持久化就停服。

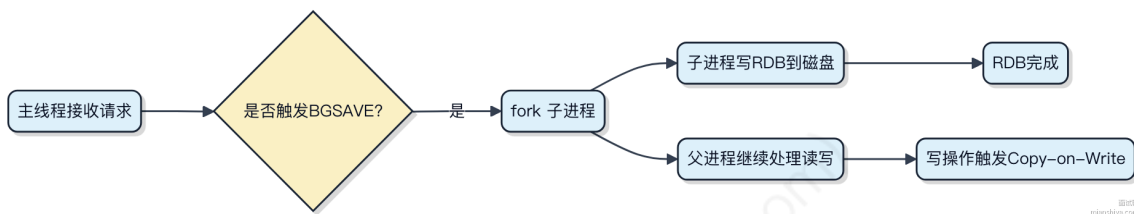
3) 但写请求会触发 **Copy-on-Write** 机制。当有 key 被修改时，操作系统才会把对应内存页复制一份给子进程，避免浪费资源。小改少复制，大改多复制，系统自动控制。

不过这里有个实际风险：如果恰好在做快照时大量 key 被修改，COW 会导致物理内存使用翻倍，极端情况可能把机器搞崩。所以一般建议避开业务高峰期做 BGSAVE，或者直接关掉手动触发，靠 AOF + RDB 混合模式来平衡。

代码上你看到的就是一个系统调用：

```
pid = fork();
if (pid == 0) {
    // 子进程执行 rdbSave()
} else {
    // 父进程继续跑 event loop
}
```

整个过程父进程基本不阻塞，除非 fork 那一瞬间要拷贝页表，数据量特别大时可能卡几十毫秒。其他时候请求压根不经过 RDB 流程，该读读该写写。



Redis 的哨兵机制是什么？

Redis 哨兵机制是为了解决主从架构下主节点故障无法自动恢复的问题。你搭了主从复制，数据能同步，但主节点挂了，整个集群就写不了了，得靠人去切主，线上系统哪经得起这种操作。

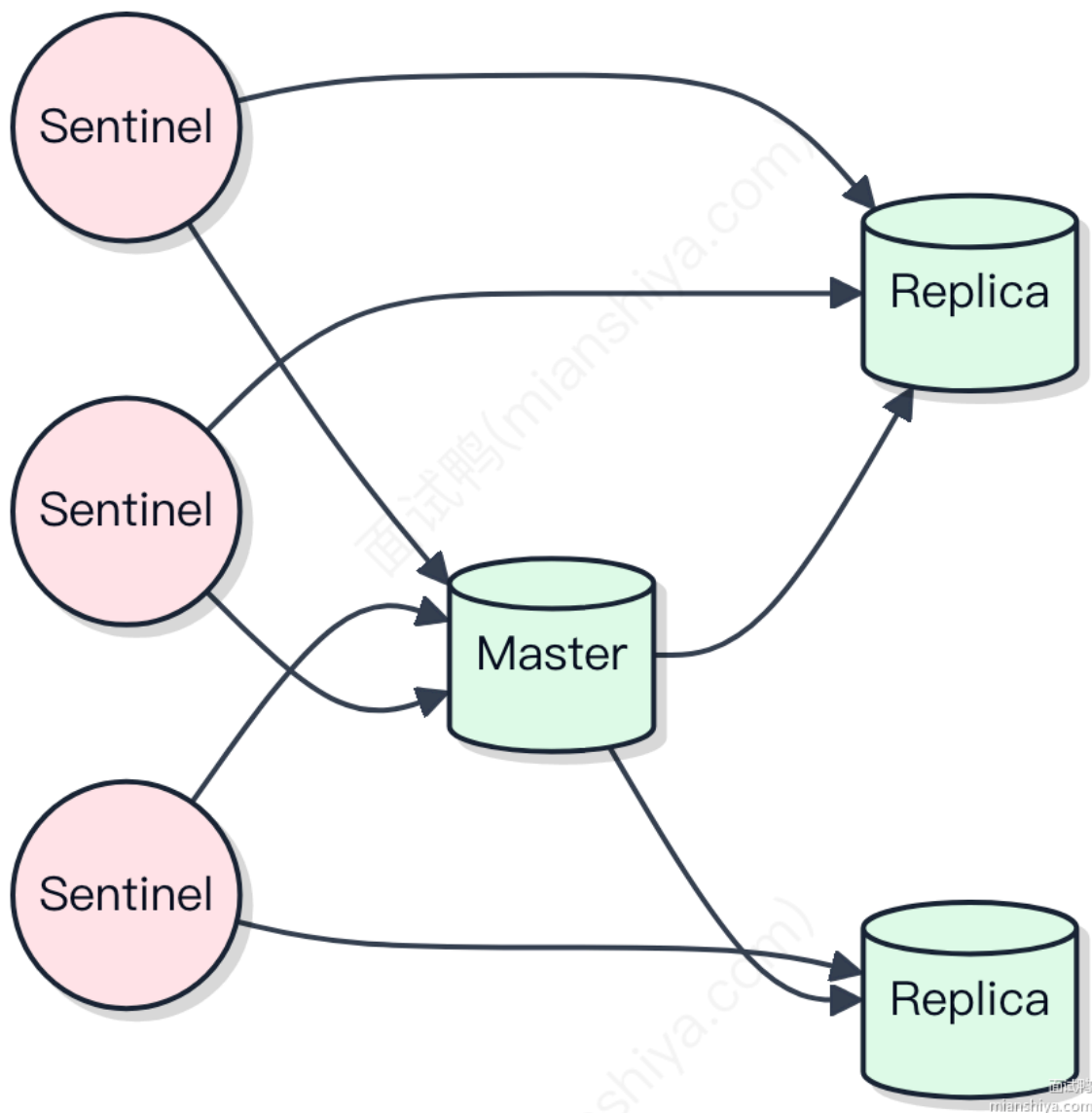
哨兵其实是独立运行的进程，一般部署 3 到 5 个，组成一个哨兵集群。它们会持续监控 Redis 主从节点的健康状态，通过心跳机制判断节点是否在线。一旦发现主节点失联，多个哨兵达成共识（比如 2/3 投票），就会触发**故障转移**流程。

这个过程分几步走：先选一个从节点提升为新主，再通知其他从节点切换主从关系，最后把旧主恢复后也变成从节点。客户端后续请求会被重定向到新主节点，服务继续可用。

代码层面，Spring Boot 项目通常配置的是 Redis Sentinel 地址列表，而不是直接写主节点。

```
@Bean
public RedisConnectionFactory connectionFactory() {
    RedisSentinelConfiguration config = new RedisSentinelConfiguration();
    config.setMaster("mymaster");
    config.setSentinels(sentinels); // 指向哨兵集群
    return new LettuceConnectionFactory(config);
}
```

整个过程不需要人工介入，高可用性就靠这套机制撑起来。不过要注意，哨兵模式还是单点写入，扩容得靠分片，比如用 Redis Cluster 才能横向扩展。



Redis 主从复制的实现原理是什么？

Redis 主从复制是靠主节点把写操作同步给从节点来实现数据一致的。整个过程分全量同步和增量同步两个阶段，大部分情况下两者会配合使用。

第一次建立复制关系时走全量同步。从节点发起 `psync` 命令带上自己的 replication ID 和 offset，主节点发现不匹配就会触发全量同步。主节点 fork 一个子进程生成 RDB 快照，同时用缓冲区记录这期间的新写命令。RDB 传完后，再把缓冲区的写命令发给从节点，确保数据完整。

之后进入增量同步阶段。主节点每执行一个写命令，都会通过网络把命令传播给所有从节点。从节点按顺序执行这些命令，保持和主节点状态一致。如果网络短暂中断，从节点可以通过 `repl_backlog_buffer` 中的积压日志快速追平，不需要重新全量复制。

心跳检测也很关键。从节点每隔 1 秒发 `REPLCONF ACK` 给主节点，主节点也能感知从节点是否存活。这种机制让复制具备一定容错能力，比如在 Redis Sentinel 或 Cluster 模式下支撑故障转移。



要注意的是，复制是异步的，从节点可能有秒级延迟。另外，默认配置下即使主节点持久化关闭，复制照样进行，这时候一旦宕机重启，从节点跟着同步空数据，会造成雪崩式数据丢失。生产环境一定要开启主节点 AOF，并设置 `min-slaves-to-write` 防止脑裂。

Redis 集群的实现原理是什么？

Redis 集群的核心是数据分片和去中心化协作。它把整个数据空间切成 16384 个槽，每个键通过 CRC16 算出哈希值后对 16384 取模，落到对应的槽上。集群里的每个主节点负责一部分槽，比如节点 A 管 05000~49999，B 管 50000~99999，C 管 100000~163839。

客户端可以直接连任意节点查数据，如果请求的 key 不归这个节点管，它会返回一个 `MOVED` 指令，告诉客户端该去哪个节点找。聪明的客户端会缓存 slot 和节点的映射关系，下次直接访问目标节点，**压根不经过转发**。

节点之间通过 Gossip 协议互相通信，每秒交换几次信息，同步自己负责的槽、健康状态、新节点加入等。这种设计没有中心节点，靠多数主节点投票完成故障转移。比如某个主挂了，它的从节点发现后会发起选举，拿到半数以上同意就能顶上。

集群也支持动态扩缩容。加新节点时先分配槽，再把旧节点上的槽迁过去，迁移过程不影响服务。可以用 `redis-cli --cluster` 工具自动完成。

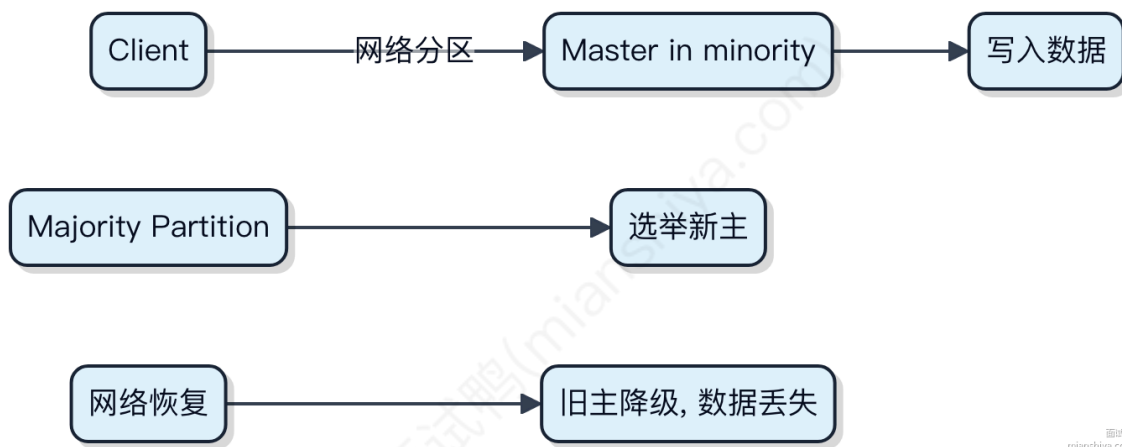
需要注意的是，集群不保证强一致性，异步复制可能导致少量数据丢失。另外多 key 操作必须都在同一个 slot，否则会报错。要用的话得靠 hash tag 强制让多个 key 落到同一槽，比如 `{user1000}.name` 和 `{user1000}.age`。

Redis 集群会出现脑裂问题吗？

Redis 集群在特定网络分区场景下，确实可能出现类似脑裂的现象，但它的设计机制能在一定程度上自动收敛和恢复。

- 1) 集群依赖多数派选举：Redis 集群采用 Gossip 协议传播节点状态，主节点故障时，从节点需要获得集群中**多数主节点的投票**才能晋升为主。如果网络分区导致某边节点数不足半数，就无法选出新主，这一侧会拒绝写入。
- 2) 小分区可能短暂失服：假设 6 节点集群（3 主 3 从），网络分成 4 节点和 2 节点两部分。4 节点这边能凑够多数派，可完成故障转移；而 2 节点那边既不能选主，也不会对外提供服务，相当于主动“自裁”，避免数据不一致。
- 3) 大分区若未隔离主：最危险的情况是，网络分区把多数节点和原主都划在同一侧，原主仍可接收写请求。而另一侧因无法达成多数派，不会升主。这时其实**不会形成双主**，只有原主在工作，不构成传统意义的脑裂。

真正风险在于客户端路由错乱。比如客户端与多数派网络断开，但仍连着一个“孤立”的主节点并持续写入。等网络恢复后，这个主的数据会被新主覆盖，造成**数据丢失**，看起来像脑裂后果。



所以严格说，Redis 集群通过多数派机制防住了双主，但无法完全避免因客户端连接滞留引发的数据不一致问题。生产环境要配合 `min-replicas-to-write` 和 `min-replicas-max-lag` 控制写安全。

Redis 的订阅发布功能是什么？你了解吗？

Redis 的订阅发布（Pub/Sub）其实就是一个基于频道的消息广播系统。生产者往一个频道发消息，所有订阅这个频道的消费者都能收到，典型的“一对多”通信。

- 1) 客户端可以 `SUBSCRIBE` 一个或多个频道，也可以用 `PSUBSCRIBE` 订阅符合模式的多个频道，比如 `news.*`。
- 2) 另一个客户端用 `PUBLISH <channel> <message>` 发消息，Redis 服务器会立刻把消息推给所有当前在线的订阅者，整个过程是**实时、同步、无持久化**的。如果消费者不在线，消息就丢了，压根不经过磁盘或队列。
- 3) 这种模式适合做即时通知类场景，比如聊天室、服务状态广播、轻量级事件驱动。但别指望它做可靠消息队列，像 Kafka 或 RabbitMQ 那种能存消息、保序、重试的能力，Redis Pub/Sub 完全搞不定。

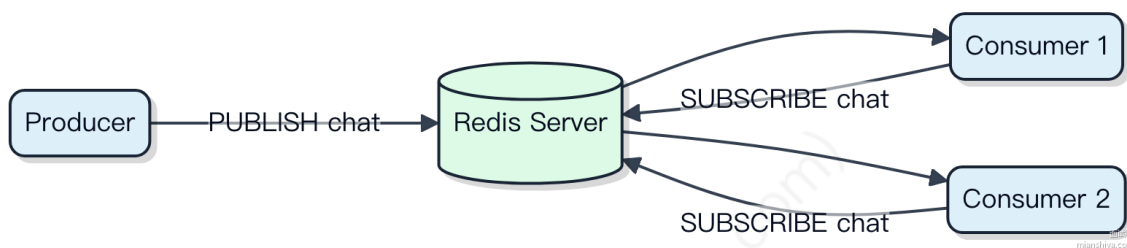
代码上很简单：

```
# 客户端 A 订阅频道
SUBSCRIBE chat_room

# 客户端 B 发布消息
PUBLISH chat_room "Hello!"
```

这时候客户端 A 会立即收到一条类型为 `message` 的响应。

要注意的是，Pub/Sub 是阻塞的，大量订阅连接会占用 Redis 线程资源。高并发下得小心使用，不然容易让 Redis 实例扛不住。有可靠性要求的场景，建议用 Redis Stream，支持消息回溯和消费组。



Redis 中如何实现分布式锁？

实现分布式锁，最朴素的想法是用一个能共享的状态来标记“锁被谁占了”，Redis 天然适合这个场景。关键不是 set 本身，而是怎么保证 set 的原子性和后续的可靠性。

1) 先说最基础的一招：SET resource_name random_value NX PX 30000

NX 是“不存在才设置”，PX 是毫秒级过期时间，random_value 要保证唯一，一般用 UUID 或线程 ID 加时间戳。这一步就能防住并发重复获取。

但光设上锁不行，释放的时候得小心。不能直接 del，不然可能删了别人的锁。正确做法是 Lua 脚本：

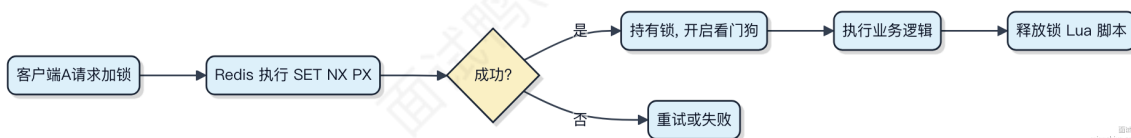
```
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
```

这个脚本保证“判断 + 删除”是原子的，值对才删，避免误删。

2) 上面这套在单 Redis 实例下够用，但主从切换时可能丢锁——主节点还没同步到从节点就挂了，新主没这把锁，别人就能抢到。这就是**锁失效**问题。

想更稳就得用 Redlock 算法，它要求跨多个独立 Redis 节点加锁，必须大多数节点成功才算拿到锁。虽然性能差些，但提高了容错性。不过实际中很多人还是用单实例 + 高可用部署，配合超时和重试扛住大部分场景。

3) 生产里常见的是 Redisson，它封装了可重入锁、看门狗自动续期。比如锁快过期了，后台线程会自动延长过期时间，只要线程还活着。这解决了业务执行时间不确定的问题。



Redis 的 Red Lock 是什么？你了解吗？

Red Lock 是 Redis 官方提出的一种分布式锁实现方案，目标是解决单实例 Redis 主从切换时锁失效的问题。它不依赖单一 Redis 节点，而是要求客户端在 **多个独立的 Redis 实例** 上申请锁，只有大多数节点都加锁成功才算真正获取到锁。

它的基本流程是这样的：

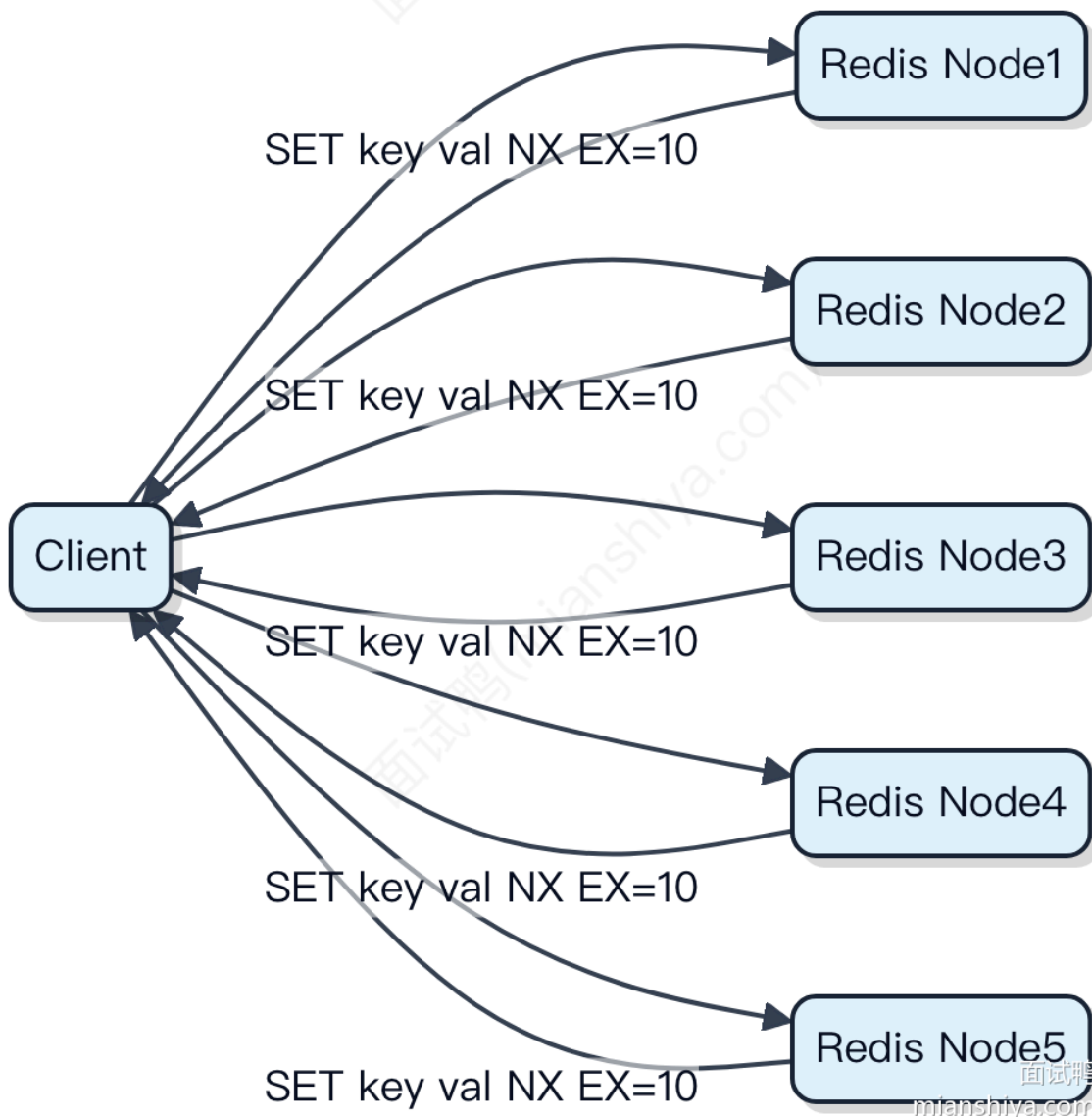
- 1) 客户端获取当前时间戳 T1
- 2) 依次向 N 个（通常是 5 个）Redis 节点发起加锁请求，使用 `SET key random_value NX EX=10` 这类命令，每个操作有较短的超时时间（比如 50ms）
- 3) 客户端获取锁结束后再次记录时间 T2
- 4) 只有当客户端在超过半数的节点上成功加锁，并且总耗时小于锁的过期时间（比如 10 秒），才认为锁获取成功

这种设计理论上能容忍最多 F 个节点故障（ $N = 2F + 1$ ）。比如部署 5 个节点，允许 2 个挂掉，依然能保证锁的安全性。

但 Red Lock 争议很大。Martin Kleppmann 等人指出，它严重依赖系统时钟的稳定性。如果某个客户端因为 GC 停顿导致时间跳跃，就可能误判锁的有效性，从而引发多个客户端同时持有同一把锁。

相比之下，像 ZooKeeper 或 etcd 这类基于共识算法（如 ZAB、Raft）的协调服务，在处理分布式锁时更安全。它们通过 leader 选举和顺序写入来保证强一致性，不需要依赖本地时钟。

代码层面，Red Lock 的核心就是多次 `SETNX + EXPIRE` 操作的组合判断，但实际生产中更推荐用 Redisson 这样的库来封装复杂逻辑。



Redis 实现分布式锁时可能遇到的问题有哪些？

Redis 实现分布式锁看着简单，其实坑不少，搞不定的话线上容易出大事。

- 1) **锁过期时间不好定**。业务执行时间不确定，设短了锁自动释放了任务还没完，别的实例就能抢锁进来，数据就乱了。设长了，万一节点宕机，锁没法释放，整个系统就卡死了。
- 2) **不是原子操作就危险**。比如先 set key 再 expire，中间 Redis 宕机，expire 没执行，锁就成“永久”了。一定要用 `SET key value NX EX seconds` 一把搞定。
- 3) **主从切换导致锁失效**。Redis 主节点写入了锁，还没同步到从节点，主节点挂了，从节点升主，新来的请求在新主上又能加同名锁——同一个资源被两个客户端同时持有锁，这叫锁的“脑裂”。
- 4) **误删别人的锁**。自己加的锁得自己删，但很多人直接 del key，如果业务超时，锁已经过期被别人获取，你这一删，就把别人的锁干掉了。正确做法是 value 用唯一标识（比如 UUID），删除前先 get 判断是不是自己的锁。
- 5) **Redlock 算法争议大**。虽然它想解决单点问题，但 Martin Kleppmann 和 Antirez 都吵过一轮，实际场景下网络延迟、GC 停顿都会让它的安全性打折扣。一般中小公司用 Redis Cluster + 带唯一值的 SET + Lua 脚本删除就够了。

真要高可靠，建议直接上 ZooKeeper 或 etcd，它们基于一致性协议，天生能保证锁的安全性，就是性能低点。Redis 锁追求的是高性能，牺牲了一定安全性，得清楚这个权衡。



Redis 中 EMBSTR 对象的阈值设置为何为 44？其调整历史是什么？

EMBSTR 是 Redis 用来优化小字符串存储的一种编码方式，它把 redisObject 和 SDS（简单动态字符串）连续存放，只用一次内存分配，减少指针和内存碎片开销。

这个 44 字节的阈值，其实是基于默认的内存分配器 jemalloc 的内存对齐规则反推出来的。Redis 在 3.2 版本引入 EMBSTR 时，规定 EMBSTR 编码的字符串最大长度为 44 字节，背后的计算是这样的：

redisObject 占 16 字节，SDS header (sdshdr8) 占 3 字节，加上字符串内容本身和末尾的 `\0`，总共不能超过 48 字节，否则 jemalloc 会从分配 32 字节的 chunk 变成 48 字节。于是留给 payload 的空间就是 44 字节： $48 - 16 - 3 - 1 = 44$ 。

- 1) 如果存的字符串长度 ≤ 44 ，Redis 用 embstr 编码，一次性 malloc 48 字节，压根不经过多次分配。
- 2) 超过 44 字节，就会退化为 raw 编码，redisObject 和 SDS 分开两次分配，管理更重。

这个值不是硬编码死的，在不同架构或分配器下可能变化。比如在一些系统上如果使用 libc malloc，可能对齐方式不同，但 Redis 为了统一行为，还是以 jemalloc 为基准定了 44。

早期版本（如 3.0）没有 embstr 这种编码，后来发现小字符串场景太多了，比如缓存 session ID、token、短配置项，像 Jedis 或 Lettuce 客户端频繁操作的小值，都集中在几十字节内，这才加了这个优化。

所以 44 不是随便拍的，是内存对齐、分配器行为和常见负载三者权衡的结果。

Redis 的虚拟内存（VM）机制是什么？

Redis 的虚拟内存不是操作系统那种虚拟内存，它其实是早期版本中一种将不常用数据从内存换出到磁盘的机制，目的是在物理内存有限时能存储更多数据。

这个功能在 Redis 2.4 版本后就被废弃了，3.0 之后彻底移除。原因很简单，引入磁盘 IO 后延迟不可控，性能下降明显，而且实现复杂，维护成本高。

现在官方推荐的方案是：1) 用更高效的 **数据分片**，比如通过 Redis Cluster 把数据分布到多个实例 2) 结合外部手段控制内存，比如设置 maxmemory + 淘汰策略 (LRU、LFU) 3) 需要持久化时交给 RDB 或 AOF，而不是运行时频繁访问磁盘

当年 VM 机制会把 value 换到磁盘，key 仍保留在内存，查找时命中 key 再去读磁盘加载 value。这听起来合理，但实际一次磁盘寻址就要几毫秒，而纯内存操作是微秒级，差了两个数量级，**延迟**直接就上去了。

代码层面你看不到 VM 相关配置了，现在配置文件里只有 maxmemory 和 maxmemory-policy 控制内存行为。

所以别在 VM 上花时间，真遇到内存撑不住的情况，优先考虑业务层做冷热分离，或者用 Redis + DB / 对象存储组合架构，这才是正解。

Redis 源码中有哪些巧妙的设计，举几个典型的例子？

Redis 的设计精巧在于它用相对简单的架构解决了高性能、高可用的难题，很多地方体现了“克制”和“恰到好处”。

SDS (Simple Dynamic String) 就是个典型。它替代了 C 原生字符串，不仅存字符串长度，还带空余空间预分配和惰性删除。获取长度是 $O(1)$ ，不会被截断，还能避免频繁内存分配。比如执行 append 时，如果剩余空间够，直接利用，不够就扩容为当前两倍。

跳表 (Skip List) 用于实现有序集合 zset。虽然平衡树更常见，但 Redis 选择跳表是因为实现简单、并发友好，平均 $O(\log N)$ 查询，且范围查询高效。像 ZRANGE 这类命令能稳定发挥，背后就是跳表在支撑。

多路复用 + 单线程模型 是它的核心取舍。一个线程处理所有请求，靠 epoll/kqueue 捕获事件，压根不经过锁竞争。命令原子执行，不存在并发修改问题。当然，大 key 或慢操作会阻塞主线程，所以生产上得避免 get 一个 100MB 的 string。

还有一个容易被忽略的细节：**dict 的渐进式 rehash**。哈希表扩容时不一次性搬数据，而是每次增删查改时迁移一个 bucket，把耗时拆开。这样避免瞬间卡顿，响应更平稳。



这种“把复杂问题分治到简单模块”的思想贯穿整个源码，比如 AOF 重写也是子进程异步完成，不影响主流程。

分布式锁在未完成逻辑前过期怎么办？

这个问题在实际用 Redis 做分布式锁时特别常见，尤其是业务执行时间不确定的时候。

1) 最直接的办法是加个**看门狗机制**。比如 Redisson 的 RLock 就自带这个功能。你设置一个锁过期时间，比如 30 秒，Redisson 会在后台起个定时任务，每隔 10 秒检查一下，如果线程还在持有锁，就自动续期到 30 秒。这样只要业务没执行完，锁就不会提前释放。

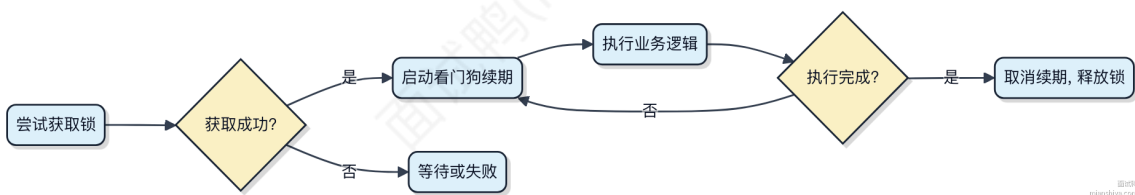
2) 另一种思路是**异步化+状态标记**。把需要加锁的长耗时操作拆出去异步执行，比如发个消息到 RabbitMQ 或 Kafka。主流程快速释放锁，消费者自己去处理任务。这时候可以用数据库或 Redis 记录任务状态，避免重复执行。

3) 如果非得同步执行，那就得**预估好超时时间**，并且做好兜底。比如通过日志或监控发现锁提前过期了，后续可以通过告警人工介入，或者设计补偿任务定期扫描未完成的操作重试。

代码上，Redisson 的用法很简单：

```
RLock lock = redisson.getLock("order:1001");
lock.lock(30, TimeUnit.SECONDS); // 自动续期
try {
    // 执行业务逻辑
} finally {
    lock.unlock();
}
```

其实关键不是锁本身，而是你要清楚：业务执行时间和锁过期时间压根不匹配的时候，光靠延长过期时间搞不定。得从整体流程设计上去应对，比如拆任务、加状态、做补偿。



Redis 中跳表的实现原理是什么？

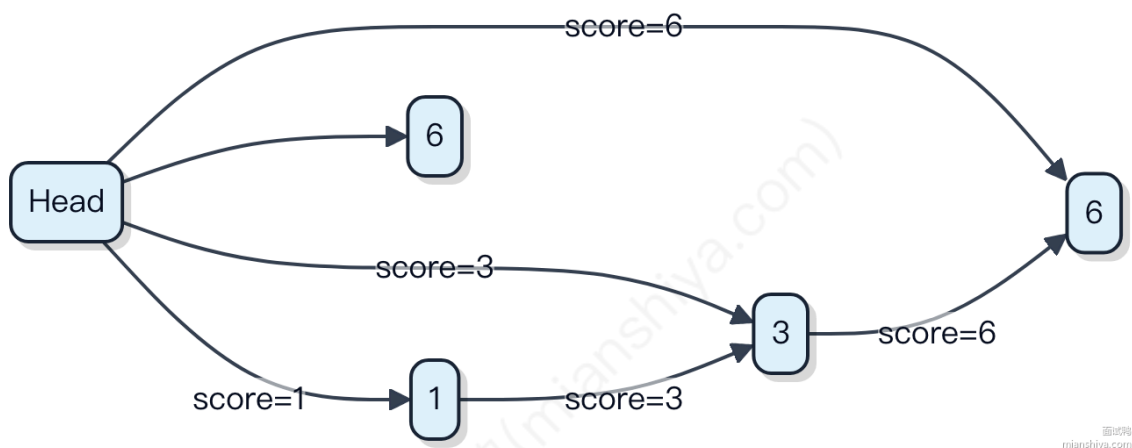
跳表（Skip List）在 Redis 里用来实现有序集合（ZSet）的底层结构，特别是当元素数量多或成员字符串较长时。它本质上是个带多级索引的链表，通过空间换时间，让查找、插入、删除操作的平均时间复杂度降到 $O(\log n)$ 。

Redis 的跳表每一层都是一个稀疏链表，最底层包含所有元素，每往上一层，节点数大概减少一半。每个节点有个随机生成的“层高”，最高不超过 32 层。查找时从顶层开始横向移动，遇到更大的就往下走，像坐电梯一样快速逼近目标。

```
// 简化版 zskiplistNode 结构
typedef struct zskiplistNode {
    sds ele;           // 成员值
    double score;       // 分值，排序依据
    struct zskiplistLevel {
        struct zskiplistNode *forward; // 指向同层下一个节点
        unsigned int span;             // 跨度，用于计算排名
    } level[];
} zskiplistNode;
```

插入新节点时，先通过随机函数决定它的层数，比如用 `rand() % 2` 不断判断是否升级，最多到 32 层。这样能保证索引分布大致均匀，避免退化单链表。

- 1) 查找快：平均只要遍历 $\log n$ 个节点就能定位
- 2) 实现比红黑树简单，没那么多旋转逻辑
- 3) 范围查询友好，比如 ZRANGE、ZRANGEBYSCORE 直接链表遍历就行



这种结构在 Redis 里扛住了大量实时排行榜场景，比如游戏积分榜、热门帖子排行，跳表的优势就在于简单高效。

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)